



<https://www.youtube.com/watch?v=hgl0p1zf31k>



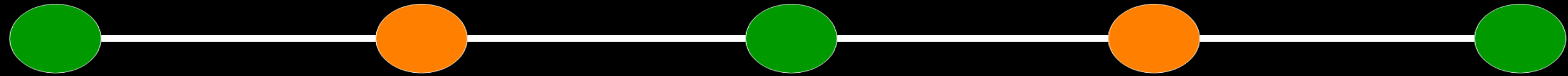
Universidad de Navarra

ESTRUCTURAS DE DATOS

Logical operators

Operator	Description
a > b	Greater than
a < b	Less than
a >= b	Greater than or equal to
a <= b	Less than or equal to
a == b	Check if value of two operands is equal
a != b	Check if value of two operands is not equal

Operator	Description
a and b	Logical AND If both operands are True then it returns True
a or b	Logical OR If one of the operands is True then it returns True
not	Logical NOT



DATA STRUCTURES



Lists, tuples and Dictionaries

Standard Data Types:

Python has 6 primitive (scalar) data types:

None

Boolean

Integer

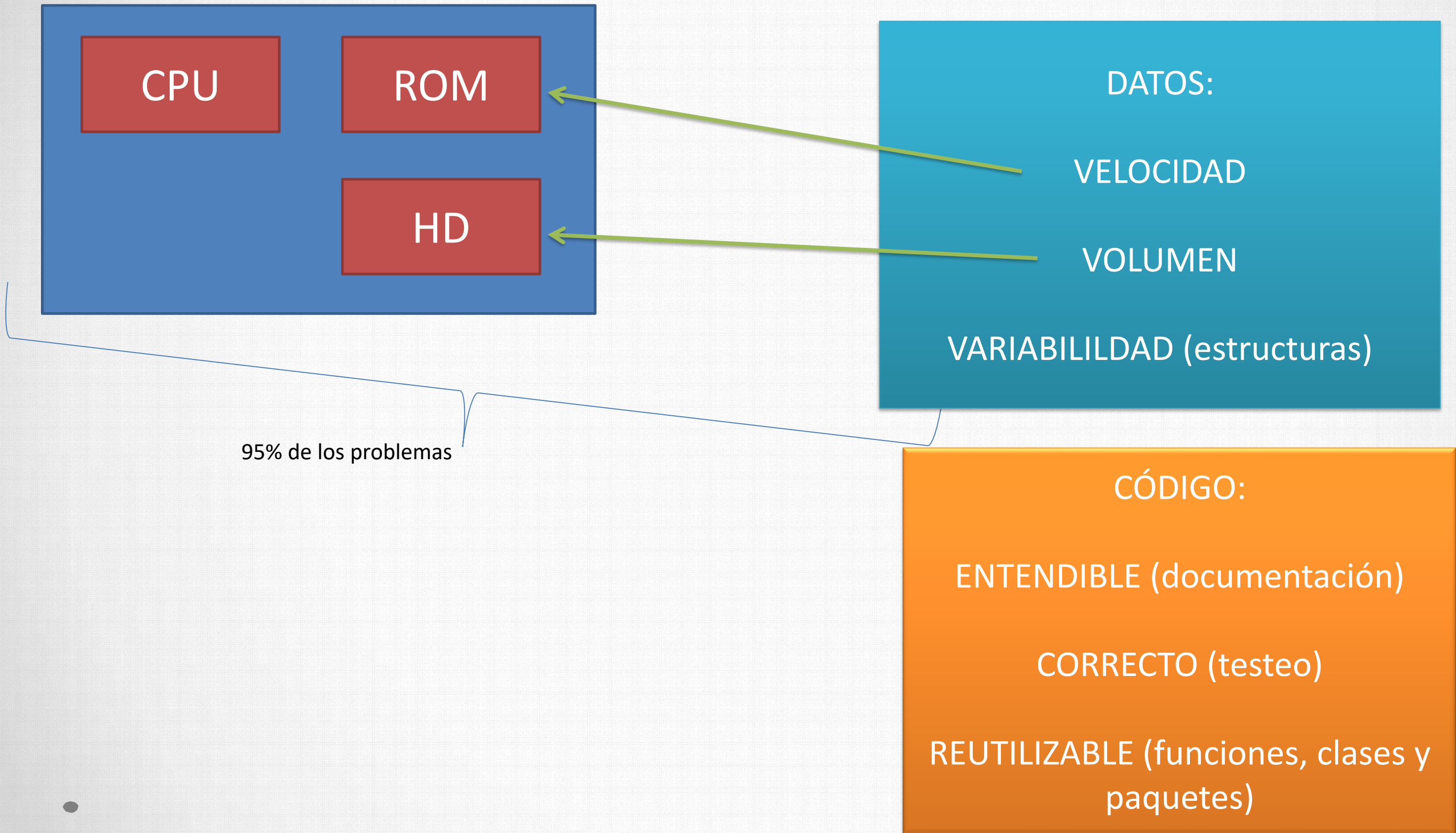
Float

Complex

String

What is a structure?

A way of organizing one or more scalar types (none, boolean, integer, float, complex and string).



Memoria principal



- Main memory en inglés.
- Es donde se almacenas los programas mientras se están ejecutando.
- También se almacenan los datos. (Von Newman)
- RAM: Random Access Memory
 - Memoria de acceso aleatorio
- Se borra cuando se apaga el ordenador
- Es muy rápida

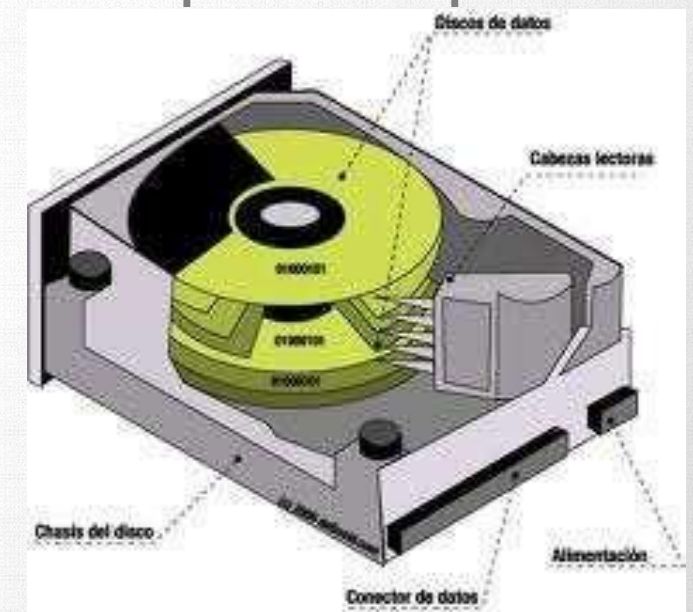
Memoria principal

- La memoria está dividida en pequeñas unidades llamadas bytes
- Los bytes están formados por bits.
- El bit podría definirse como la unidad mínima de información.
- A cada byte en la memoria se le asocia una dirección.



Almacenamiento secundario

- Es un tipo de memoria que almacena datos durante mucho tiempo.
- No se borran cuando se apaga el ordenador.
- Es más lenta que la memoria principal
- Frecuentemente, los programas son almacenados en la memoria secundaria y cargados en la memoria principal cuando es necesario.
- Ej: Discos duros, SSD, etc.



Built-in Data Structures:

Python has 4 standard data structures:

List

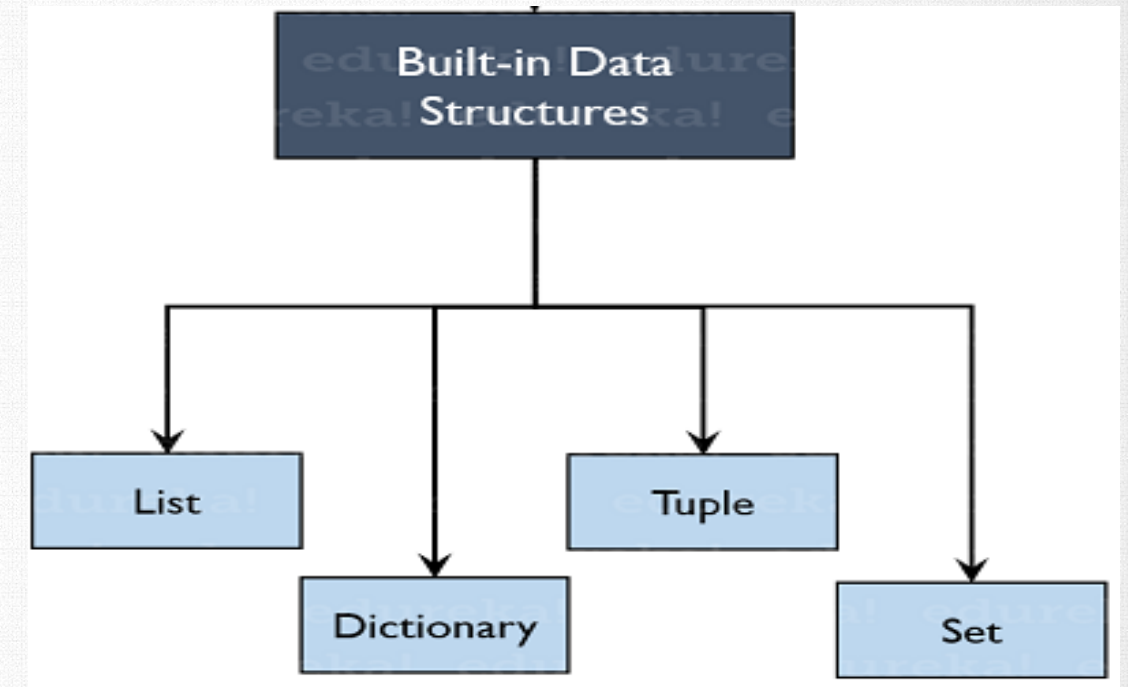
Tuple

Dictionary

Set

What is a structure?

A way of organizing one or more scalar types (none, boolean, integer, float, complex and string).



Python Lists



Python Lists:

Lists are the most versatile of Python's compound data types. A list contains items separated by commas and enclosed within square brackets ([]).

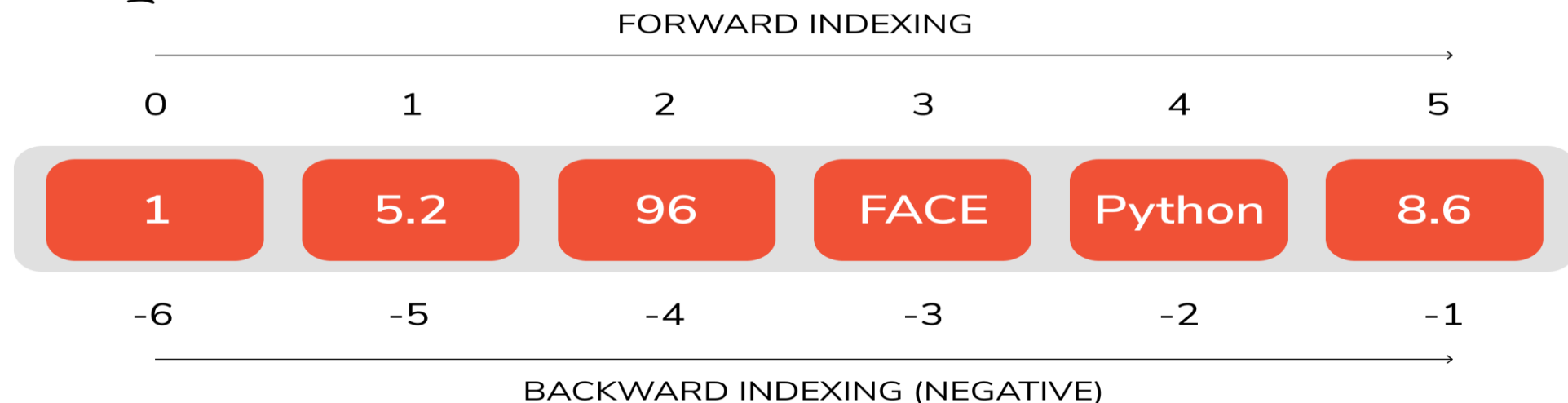
To some extent, lists are similar to lists in R. However lists are the default option in Python. All the items belonging to a list can be of different data type.

The values stored in a list can be accessed using the slice operator ([] and [:]) with indexes starting at 0 in the beginning of the list and working their way to end-1.

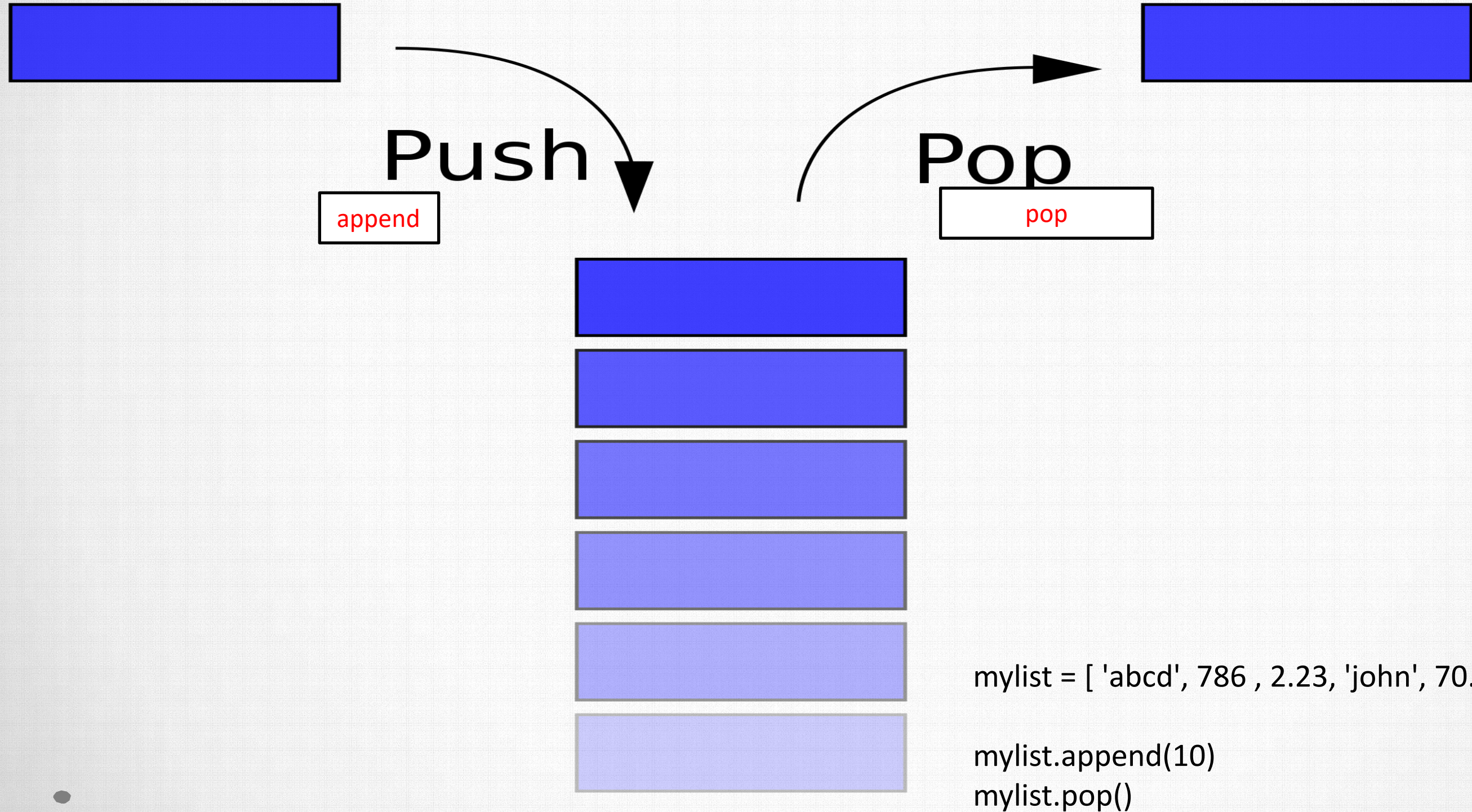
The plus (+) sign is the list concatenation operator. and the asterisk (*) is the repetition operator.

List = [1, 5.2, 96, "FACE", "Python", 8.6]

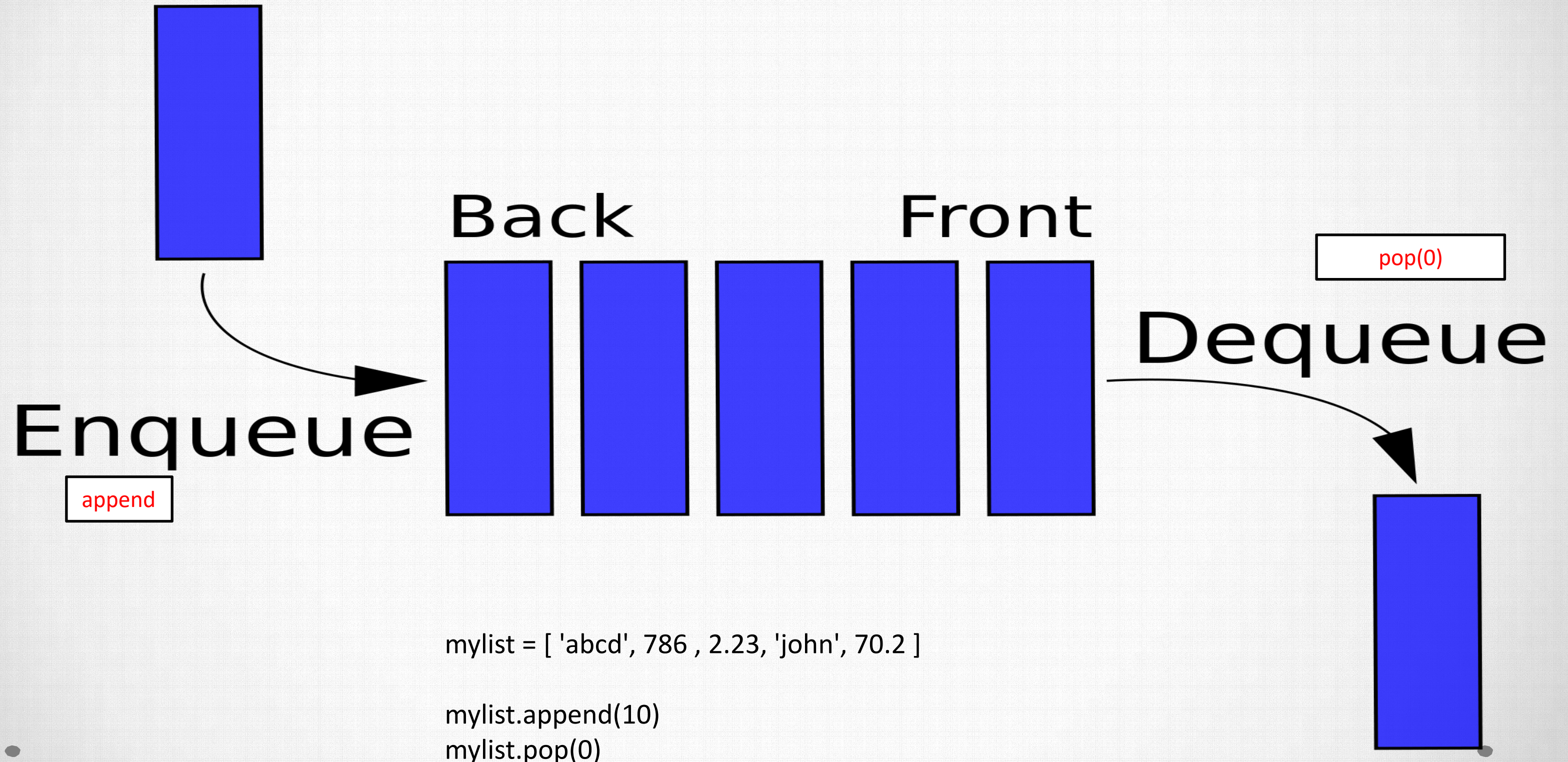
View of a list
→

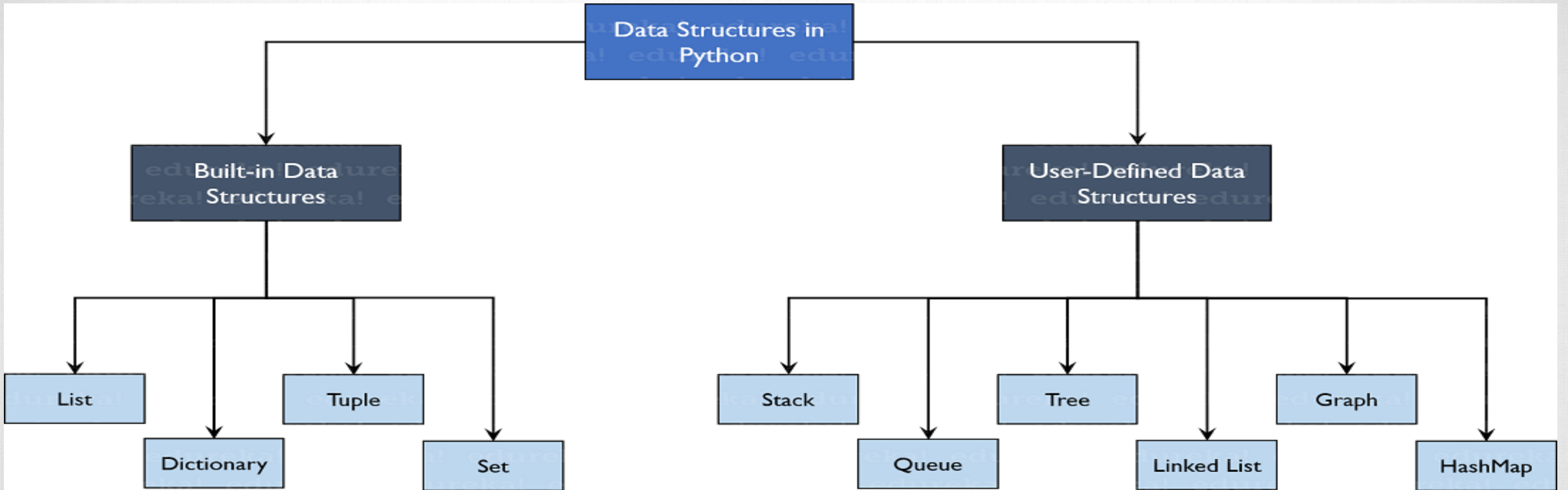


List as stack (Last In First Out)



List as queue (First In First Out)





Escribir un programa en
Python usando un loop for
para sumar todos los
elementos de la lista:

```
mi_lista = [1, 2, 3, 4, 5]
```

Solución:

```
# Definir la lista de números  
mi_lista = [1, 2, 3, 4, 5]
```

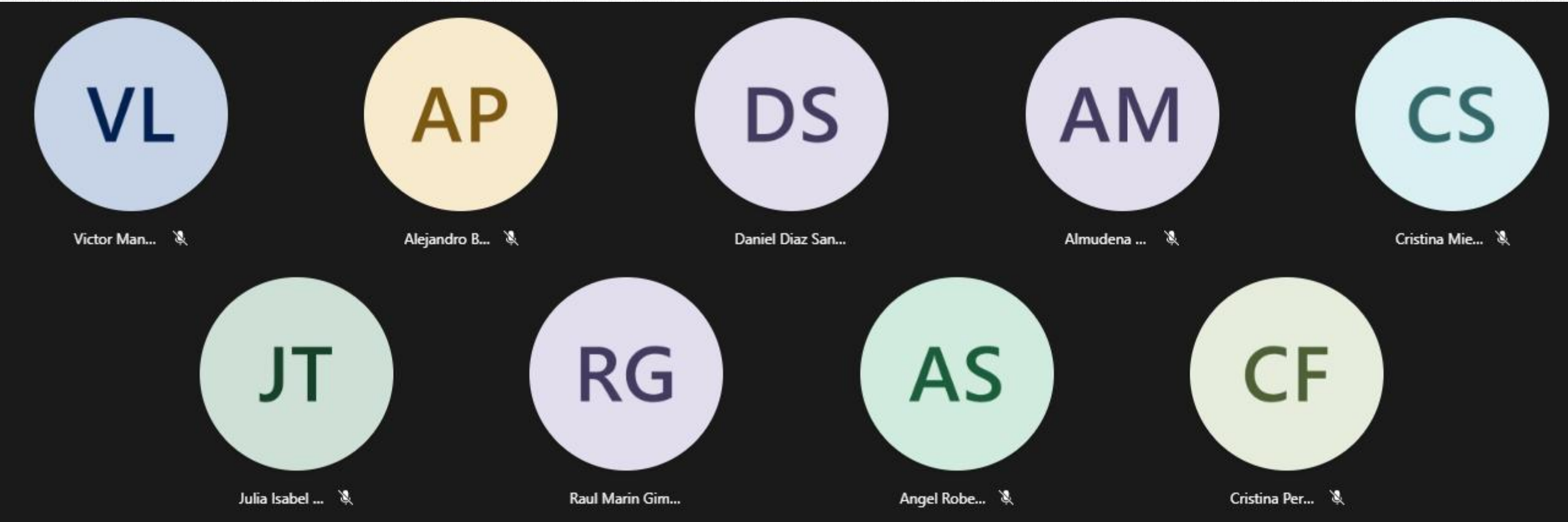
```
# Inicializar una variable para guardar la suma  
suma = 0
```

```
# Recorrer cada elemento de la lista  
for x in mi_lista:  
    # Sumar el elemento a la variable suma  
    suma += x
```

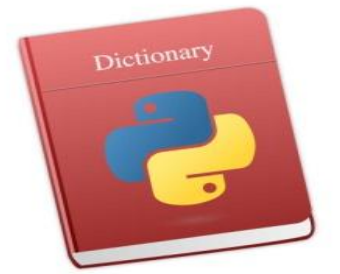
```
# Mostrar el resultado  
print(suma)
```

Python Dictionaries





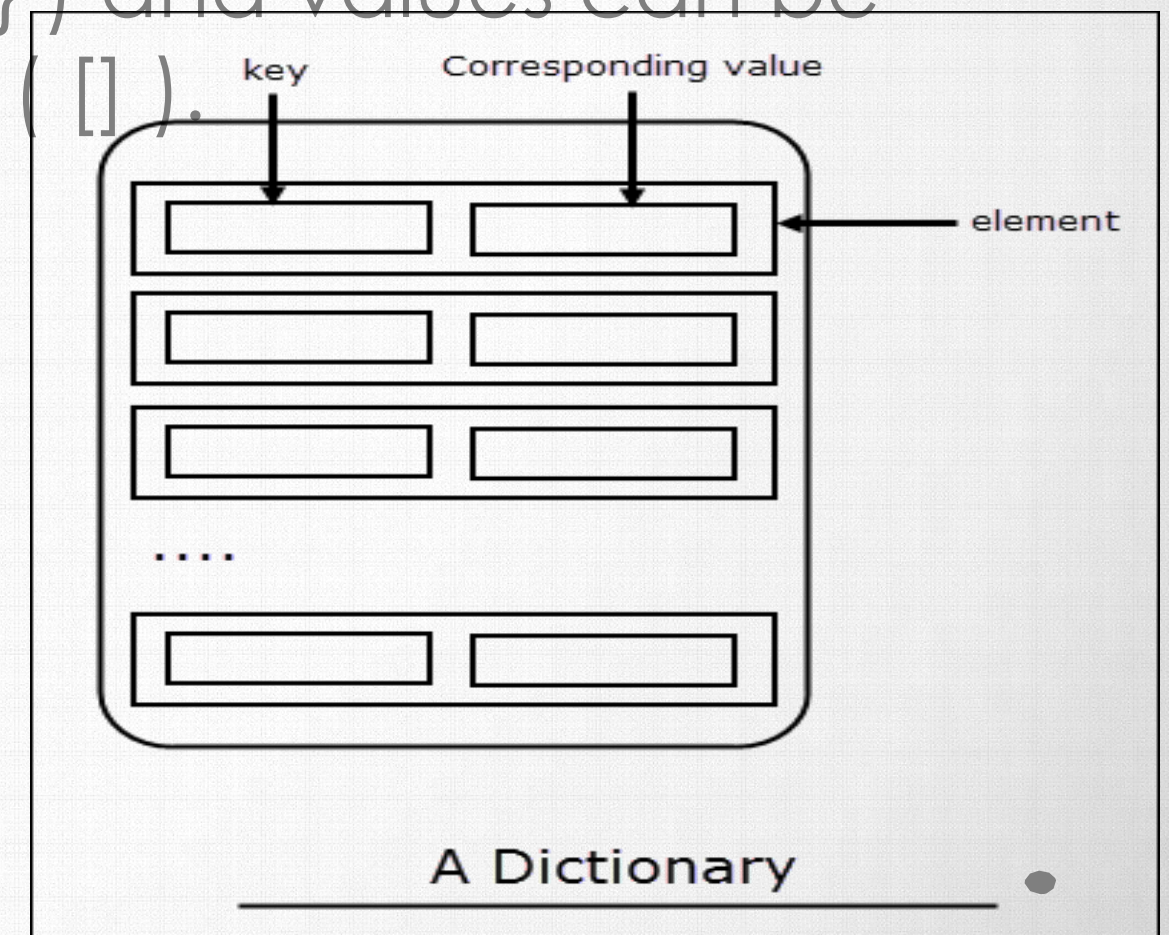
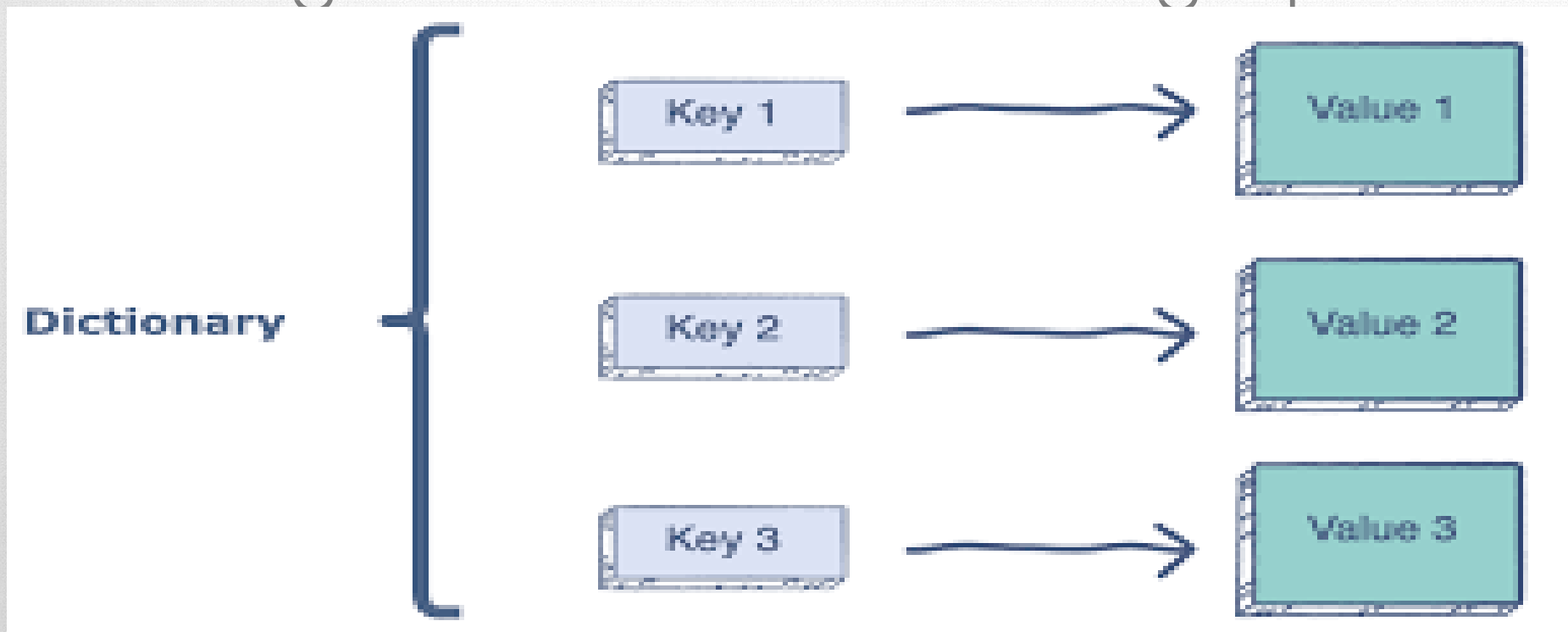
Python Dictionary:



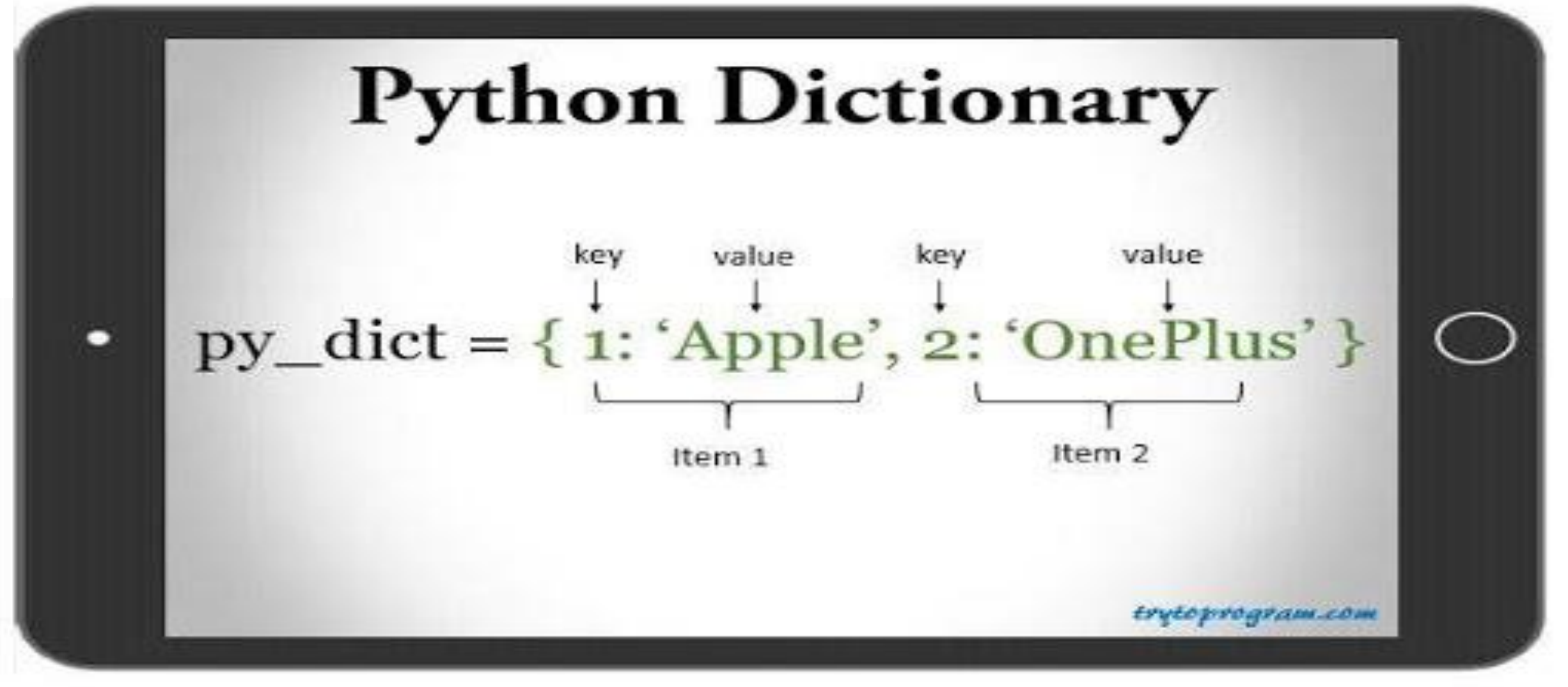
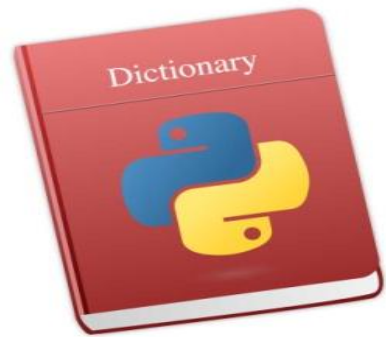
Python's dictionaries are hash table type. They work like associative arrays or hashes found in Perl and consist of key-value pairs.

Keys can be almost any Python type, but are usually numbers or strings. Values, on the other hand, can be any arbitrary Python object.

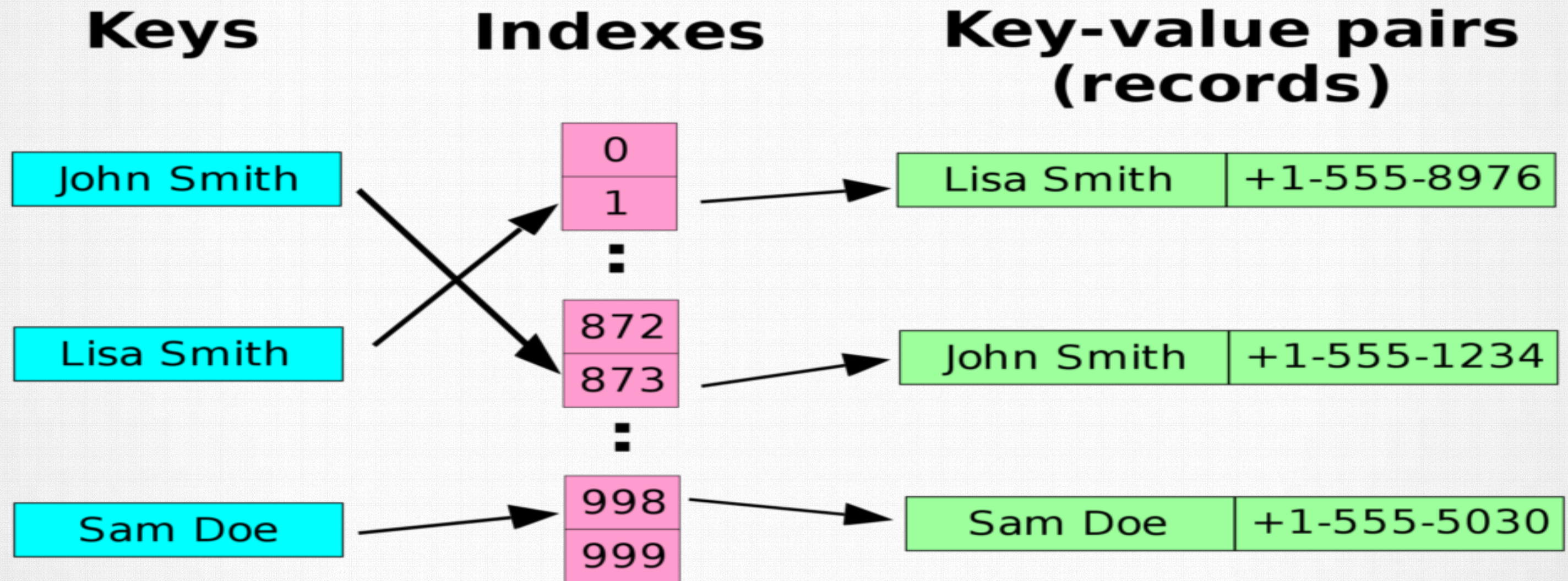
Dictionaries are enclosed by curly braces (`{ }`) and values can be assigned and accessed using square braces (`[ ]`).



Python Dictionary - Syntax



Python Dictionary – Is it efficient?



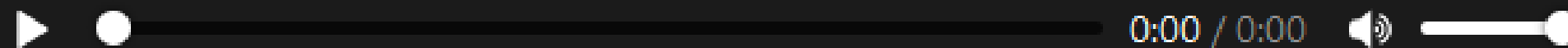
The fastest structure to find an element by key!

Python past, present, and future with Guido van Rossum

Episode #100, published Wed, Feb 22, 2017, recorded Wed, Jan 18, 2017.



Guido van Rossum



Which of the following statements are true?

- a) Dictionaries do not have much value in Python
- b) Dictionaries are a fundamental part of Python
- c) Dictionaries are used extensively in applications
- d) Dictionaries are not at par with the tech innovation

Correct answer at 49:10

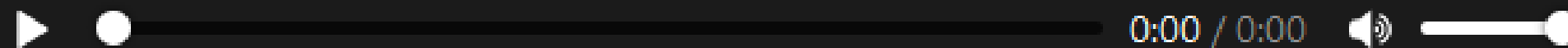
<https://talkpython.fm/episodes/show/100/python-past-present-and-future-with-guido-van-rossum>

Python past, present, and future with Guido van Rossum

Episode #100, published Wed, Feb 22, 2017, recorded Wed, Jan 18, 2017.



Guido van Rossum



Which of the following statements are true?

- a) Dictionaries do not have much value in Python
- b) Dictionaries are a fundamental part of Python
- c) Dictionaries are used extensively in applications
- d) Dictionaries are not at par with the tech innovation

Correct answer: b and c at 49:10

<https://talkpython.fm/episodes/show/100/python-past-present-and-future-with-guido-van-rossum>

50% Good / Appropriate Algorithm	30% Good / Appropriate Data Structure	20% Faster / +Memory Computer
---	--	--

Like Constants!

Pure Python
(comes within the
language)

	list	tuple	dictionary	set
COMMON:		Collection of 0, 1 or many of any combination of types of Python scalars		
DIFFERENTIAL:	Mutable, Ordered [0,n-1]	Immutable, Ordered [0,n-1]	Key is Unique and Unordered (key immutable / Value = Mutable)	Unique, Unordered
CREATE EMPTY:	list1=list(); list2=[]	tuple1=tuple(); tuple2=()	dict1=dict(); dict2={}	set1 = set()
CREATE:	list1 = [3,6,7,3,7]	tupple1 = (3,3,6,7,7)	dict1 ={3:'str',6:False}	set1 = {3,6,7}
INSERT:	list1.append(8)	-	dict1['Juan'] = 'Casado'	set1.add(8)
INSERT IN POSITION:	list1.insert(3,'value')	-	-	-
REMOVE:	list1.pop(1) / list1.remove(3)	-	del dict1['Juan']	set1.remove(8)

Python
Ecosystem
(libraries)

Data frame (pandas)

Key-Value (pickledb)

HDFS (pytables)

SQL

Python Basic Exercises 5 - Lists

(home work: will be solved next class)

- **Write a program which accepts a sequence of comma-separated numbers from console and generate a list and a tuple with the square of every number.**
- **Suppose the following input is supplied to the program:
34,67,55,33,12,98**
- **Then, the output should be:
[1156, 4489, 3025, 1089, 144, 9604]
(1156, 4489, 3025, 1089, 144, 9604)**

Hint: In case of input data being supplied to the question, it should be assumed to be a console input. tuple() constructor method can convert list to tuple

Python Basic Exercises 5 - Lists

(home work: will be solved next class)

- **Write a program which accepts a sequence of comma-separated numbers from console and generate a list and a tuple with the square of every number.**

SOLUTION:

```
numbers_str = input("Please enter a comma-separated sequence of numbers:")
numbers_list = numbers_str.split(",")

numbers_squared_list = []
for item in numbers_list:
    numbers_squared_list.append(float(item)**2)

numbers_squared_tuple = tuple(numbers_squared_list)
print(numbers_squared_list)
print(numbers_squared_tuple)
```

Python Basic Exercises 6 - Dictionary

(home work: will be solved next class)

- **With a given integer number n, write a program to generate a dictionary that contains (i, i*i) such that i is an integral number between 1 and n (both included). Finally, the program should print the dictionary.**
- **Suppose the following input is supplied to the program: 8
Then, the output should be:
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64}**

Hint: In case of input data being supplied to the question, it should be assumed to be a console input. Consider using the constructor method dict()

Python Basic Exercises 6 - Dictionary

(home work: will be solved next class)

- With a given integer number n , write a program to generate a dictionary that contains $(i, i*i)$ such that i is an integral number between 1 and n (both included). Finally, the program should print the dictionary.

SOLUTION:

```
n = int(input("Please enter a positive integer number: "))

n_dict = {}
for i in range(n):
    n_dict[i] = i**2

print(n_dict)
```

list: delete items from the end of a list called myList: myList.pop()

dictionary: delete a key-value pair from the high_lows dictionary:
del high_lows[key]

set: delete an item from a set: myset.remove(item)

Example:

```
# language set
language = {'English', 'French', 'German'}
# removing 'German' from language
language.remove('German')
# Updated language set
print('Updated language set:', language)
```

About del high_lows[key]

```
del something # (1) It is procedural programming
del(something) # (2) It is functional programming
something.del() # (3) It is object oriented programming
```

3 different paradigms, and Python is 80% (3), 19% (2)³² and some bizarre occasions it is (1) like in del high_lows[key]!!!

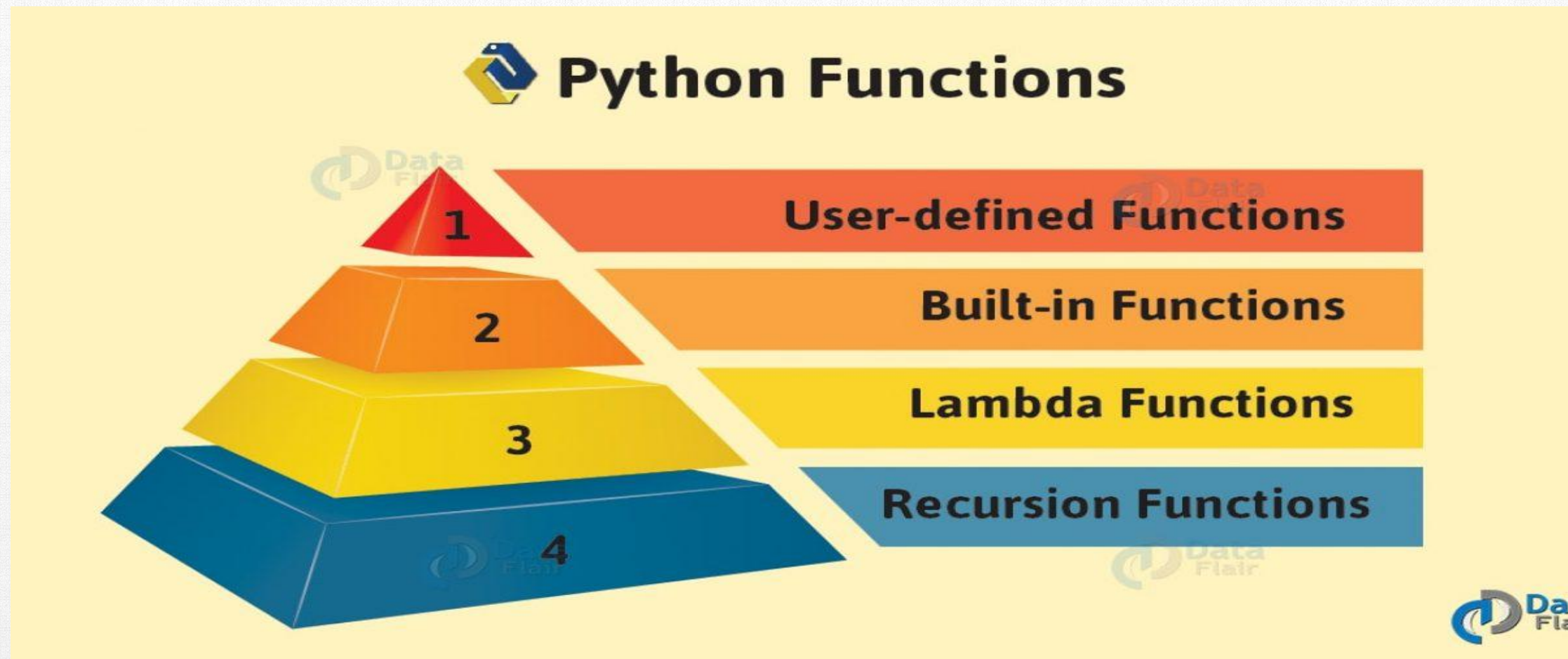
Normally (3) means that the function is specific to the object therefore called **method**, (2) means that a function can be applied to more than one type of object and (1) it means that Python has to call low level operations to do the trick.



Universidad de Navarra

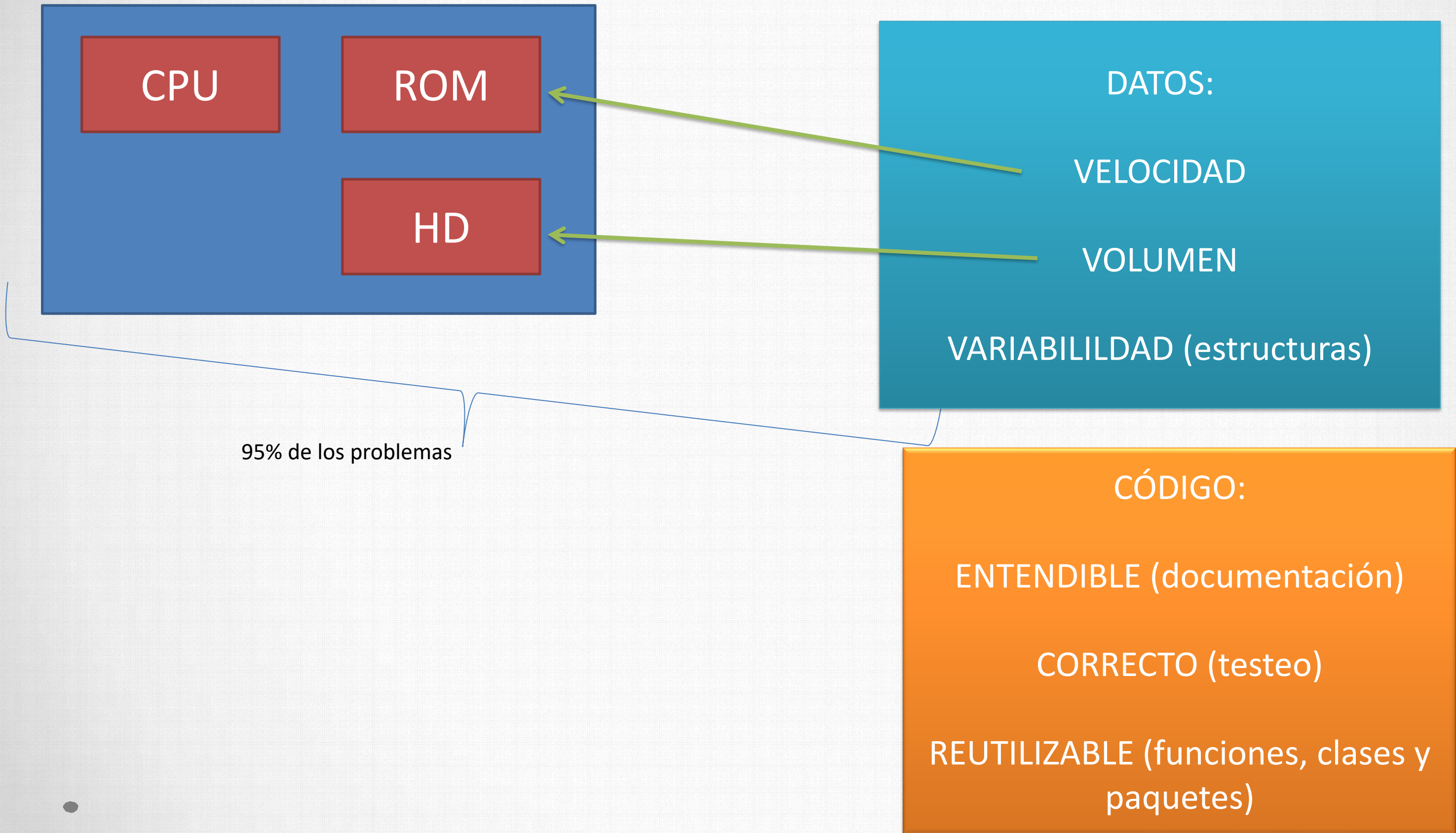
Functions

a named section of a program that performs a specific task



Functions:

A way of organizing code to make it more understandable, easy to use, reproducible and easier to share.



list: delete items from the end of a list called myList: myList.pop()

dictionary: delete a key-value pair from the high_lows dictionary:
del high_lows[key]

set: delete a item from a set: myset.remove(item)

Example:

```
# language set
language = {'English', 'French', 'German'}
# removing 'German' from language
language.remove('German')
# Updated language set
print('Updated language set:', language)
```

About del high_lows[key]

```
del something # (1) It is procedural programming (PP)
del(something) # (2) It is functional programming (FP)
something.del() # (3) It is object oriented programming (OOP)
```

3 different paradigms, and Python is 80% (3 - OOP), 19% (2 - FP) and some bizarre cases it is (1 - PP) like in del high_lows[key]!!!

Normally (3) means that the function is specific to the object therefore called **method**, (2) means that a function can be applied to more than one type of object and (1) it means that Python has to call low level operations to do the trick.

functions vs. methods

A function looks like this: `function(something)`

And a method looks like this: `something.method()`

```
>>> a = ' BiG MuG '  
>>> a.lower()  
'big mug '  
>>> a.upper()  
'BIG MUG '  
>>> a.strip()  
'BiG MuG'  
>>> a.strip().upper()  
'BIG MUG'  
>>> a.strip().upper().split(' '  
['BIG', 'MUG']  
>>> ';'.join(a.strip().upper().split(' '  
'BIG;MUG'
```

A function can be applied to one or more type of things (objects), while methods are specific and can only be applied to “things” (objects) of one given type (one given class)

User defined functions



1 `def` 2 `function_name` 3 `(args)` 4 `:`
5 `#code in function`

```
(bottom)
1  def shake():
2      print('bugaloo')
3      ...
4  shake()
5
```

Function & body

Function call

Debug I/O Exceptions Python Shell Messages OS Commands

Debug I/O (stdin, stdout, stderr) appears below

```
bugaloo
```

Function output

Line 5 Col 0 - [User]

```
#### FUNCTIONS AND NOTHING - None is a type in python
#### FUNCIONES Y LA NADA - None es un tipo en python
def func1():
    """ this is my own doc string - it becomes available to anyone who calls help(func1). """
    pass

if func1()==None:
    print("It returned the 'nothing'! / Me ha devuelto la 'nada'!")
else:
    print("It returned something!")

help(func1)
```

```
#### FUNCTIONS AND NOTHING - None is a type in python
#### FUNCIONES Y LA NADA - None es un tipo en python
def func1():
    """ This is my own doc string - it becomes available to anyone who calls help(func1)."""
    pass

if func1()==None:
    print("It returned the 'nothing'! / Me ha devuelto la 'nada'!")
else:
    print("It returned something!")

help(func1)
```

Numeric
Strings
Constructors
Iterables (Lists, Dicts,...)

Built-in Functions

Python Functioning
Input / Output

abs() – Numeric

delattr()

hash()

memoryview()

set() - Constructors

all()

dict() - Constructors

help()

min() - Numeric

setattr()

any()

dir()

hex() - Constructors

next()

slice()

ascii()

divmod()

id()

object()

sorted()

bin()

enumerate()

input() - Input / Output

oct()

staticmethod()

bool() - Constructors

eval() - Python Functioning

int()

open() - Input / Output

str()

breakpoint()

exec()

isinstance()

ord()

sum()

bytearray()

filter()

issubclass()

pow()

super()

bytes() - Constructors

float()

iter()

print() - Input / Output

tuple()

callable()

format() - Strings

len()

property()

type()

chr() - Strings

frozenset()

list() - Constructors

range() - Iterables

vars()

classmethod()

getattr()

locals()

repr()

zip()

compile() - Python Functioning

globals()

map()

reversed() - Strings

__import__()

complex() - Constructors

hasattr()

max() - Numeric

round()

```
#### FUNCTIONS - DEFAULT PARAMETERS & RECURSION- THE ZEN OF PYTHON.  
#### FUNCIONES - PARAMETROS DEFAULT & RECURSIÓN - EL ZEN DE PYTHON.  
def func2(y=True,z_lista=[], x=10, recursion_level=1):  
    if (recursion_level<2):  
        func2(x,y,z_lista.copy(),recursion_level+1)  
    d_lista[2] = recursion_level  
    return(x,y,z_lista)  
  
a,b,c=func2()  
print(a,b,c)  
  
a,b,c=func2(True,d_lista.copy(),12,recursion_level=0)  
print(a,b,c)
```

```
#### FUNCTIONS - DEFAULT PARAMETERS & RECURSION- THE ZEN OF PYTHON.  
#### FUNCIONES - PARAMETROS DEFAULT & RECURSIÓN - EL ZEN DE PYTHON.  
def func2(y=True,z_lista=[], x=10, recursion_level=1):  
    if (recursion_level<2):  
        func2(x,y,z_lista.copy(),recursion_level+1)  
    d_lista[2] = recursion_level  
    return(x,y,z_lista)  
  
a,b,c=func2()  
print(a,b,c)  
  
a,b,c=func2(True,d_lista.copy(),12,recursion_level=0)  
print(a,b,c)
```

```
##### TUPLES & FUNCTIONS - PARAMETERS and RETURN VIA TUPLES
##### FUNCIONES - PARAMETROS y RETURN VIA TUPLAS.
d_list = [1,"a",True]

# On the right side you are packaging in a tuple.
# / En la parte derecha está empaquetando en una tupla.
# On the left side you are unpacking the tuple into variables.
# / En la parte izquierda está desempaquetando de la tupla en variables.
(d_list[2], d_list[1], d_list[0]) = (d_list[0], d_list[1], d_list[2])

d_tuple = (1,"a",True)
d_list[1] = "ready!"
print(d_list[1],d_tuple[1])

def func2(x,y,z):
    return(x,y,z)

a,b,c=func2(12,True,"Pepe")
print(a,b,c)
```

```
d_list = [1,"a",True]
(d_list[2], d_list[1], d_list[0]) = (d_list[0], d_list[1], d_list[2])
```

```
d_tuple = (1,"a",True)
d_list[1] = "ready!"
print(d_list[1],d_tuple[1])
```

```
def func2(x,y,z):
    return(x,y,z)
```

```
a,b,c=func2(12,True,"Pepe")
print(a,b,c)
```

for winning an extra participation you have to send your solution for the `soco_bate_vira(n)` extra participation Python function challenge by email until Monday 24th of May 23:59.

Please send straight as text in the body of the email to `mfalonso@faculty.ie.edu` (do not include as a `soco_bate_vira.py` as it slows down me checking it, thanks!)

Find the pattern, then write a function: `soco_bate_vira(n)`

That receives:

```
> soco_bate_vira(2)
```

```
[1] Soco Soco Bate Bate Soco Soco Vira Vira
```

```
> soco_bate_vira(1)
```

```
[1] Soco Bate Soco Vira
```

```
> soco_bate_vira(0)
```

```
[1] Soco Bate Vira
```

What is the result of `soco_bate_vira(3)`?

```
soco_bate_vira(4)
```

```
soco_bate_vira(5)
```

```
soco_bate_vira(N)
```

```
soco_bate_vira(100)
```

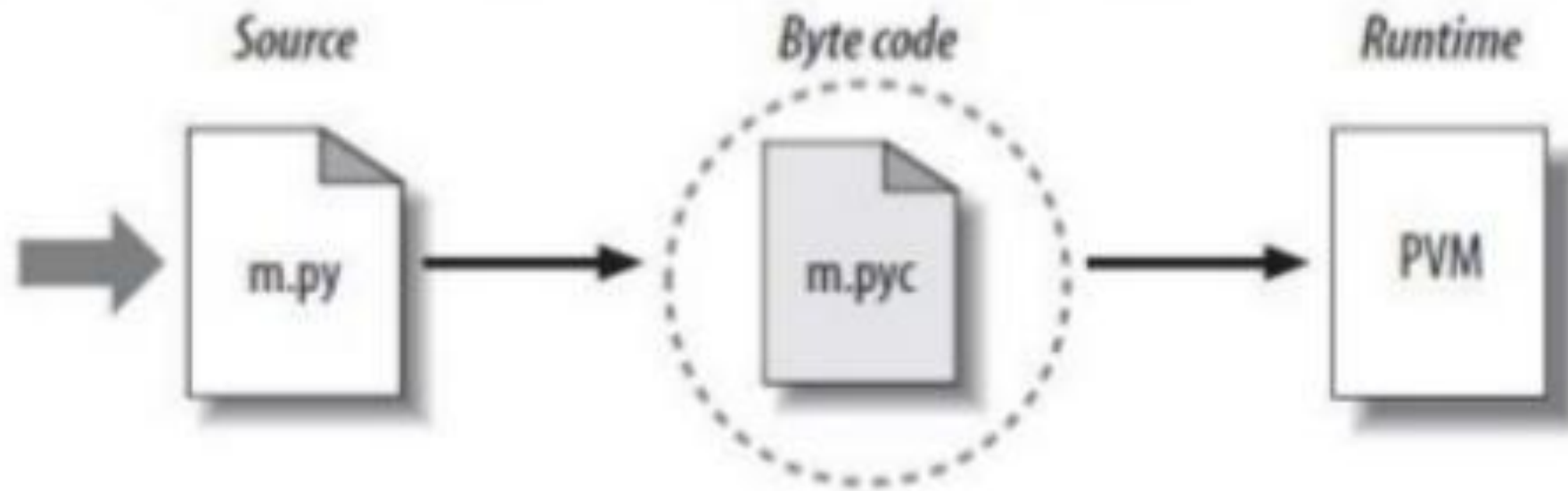
https://www.youtube.com/watch?v=VZGcm_JYOKM



MATERIAL EXTRA

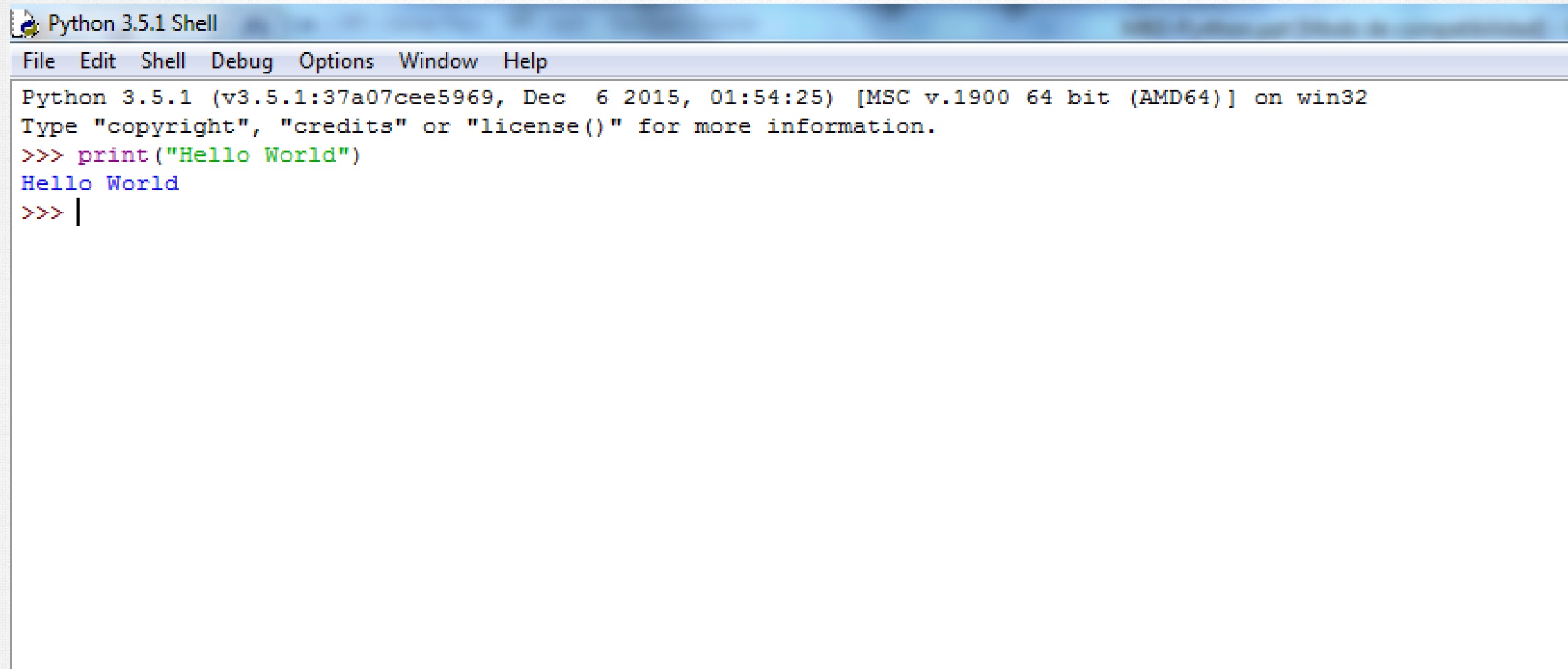
Python

Python is interpreted



Source code extension is **.py**
Byte code extension is **.pyc** (compiled python code)

Scripting Python

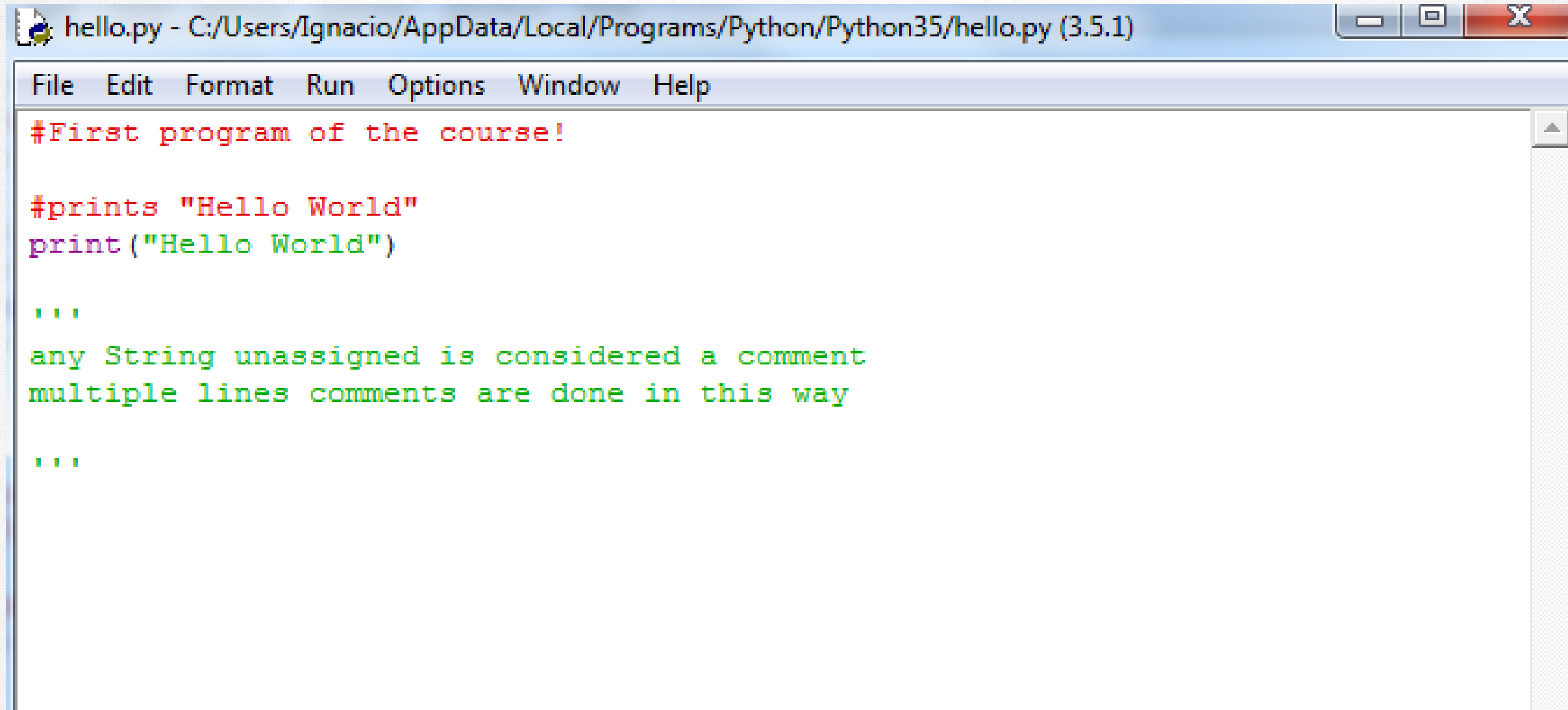


```
Python 3.5.1 Shell
File Edit Shell Debug Options Window Help
Python 3.5.1 (v3.5.1:37a07cee5969, Dec 6 2015, 01:54:25) [MSC v.1900 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> print("Hello World")
Hello World
>>> |
```



Back to the theory...

Comments in Python



The screenshot shows a window titled 'hello.py - C:/Users/Ignacio/AppData/Local/Programs/Python/Python35/hello.py (3.5.1)'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code editor contains the following Python code:

```
#First program of the course!

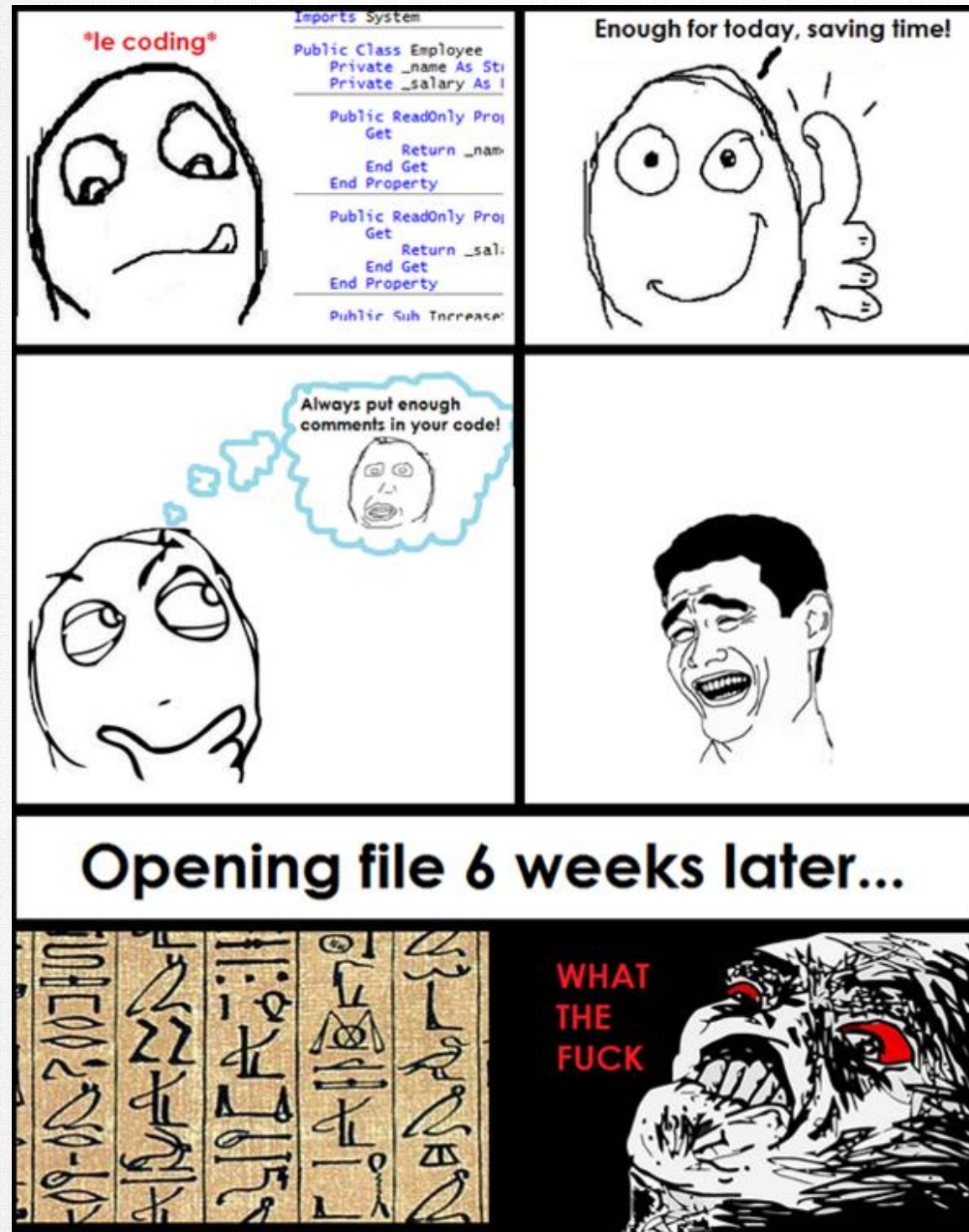
#prints "Hello World"
print("Hello World")

...
any String unassigned is considered a comment
multiple lines comments are done in this way
...
```

for a single line comment

"""...""" for a multiple line comment

Comments in Python



- Comment your code
- Think what will happen when you open the source file three months after coding it...
- General overview of the program/function/class
- Important variables/calculations
- Design considerations

String escape sequences

Escape Sequence	Output
\\	Backslash
\'	Single quote
\"	Double quote
\b	ASCII Backspace
\n	Newline
\t	Tab
\u12af	Unicode 16 bit
\U12af89bc	Unicode 32 bit
\o84	Octal character
\xFF	Hex character

Variables in Python

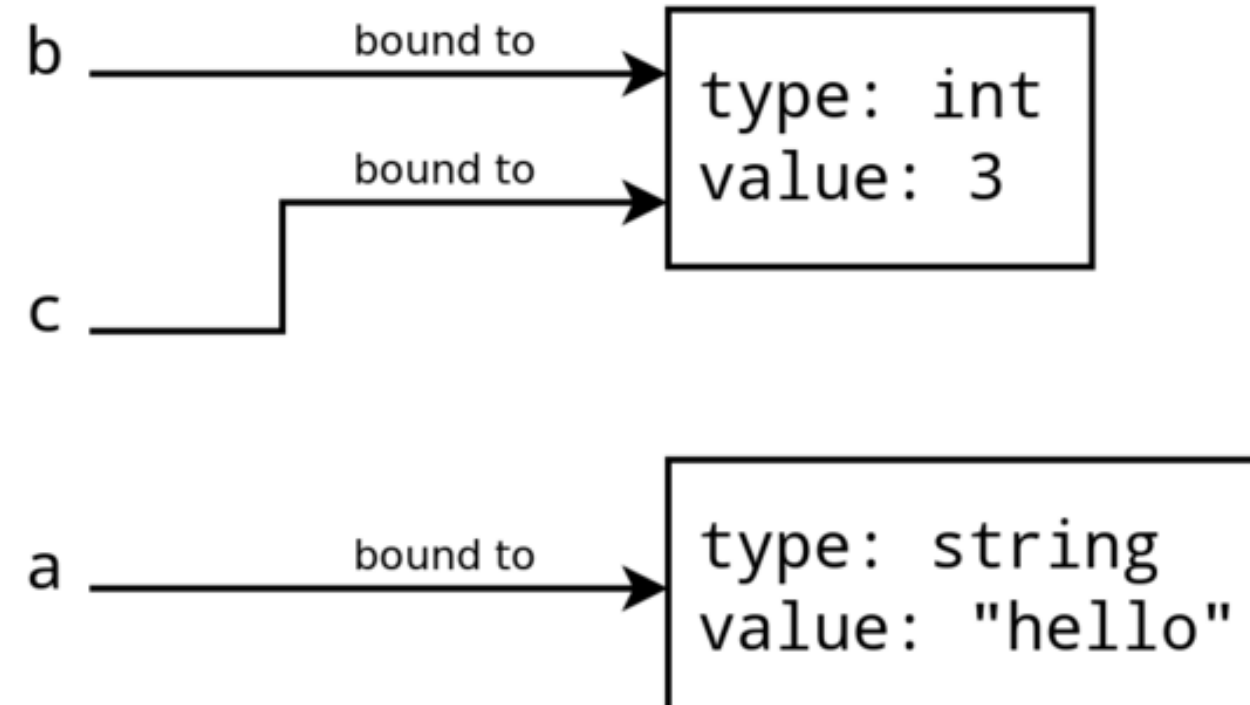
Executed Code:

Variable Assignment

```
a = 3  
b = a  
c = a  
a = "hello"
```

Variables

Values in Memory



Variables in Python

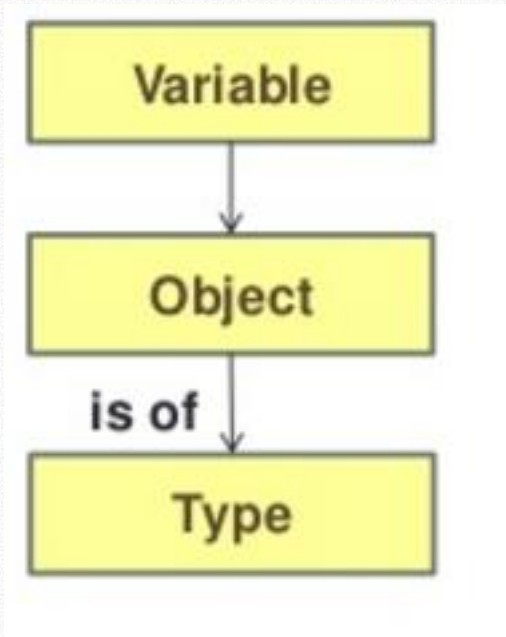
- Don't need to have a pre-defined type, they get the type from the value they are pointing at
- Four main types:
 - String – holds text based values
 - name = "Ignacio"
 - int – Integer numbers
 - name = 10
 - float – floating decimal numbers
 - name = 10.4
 - Boolean (True or False)
- More variable types (lists, tuples, dictionaries to be reviewed during the course)

Variables in Python

- A variable has
 - a name (identifier)
 - a type
 - a scope
 - and ...a value
- A valid identifier is a non-empty sequence of characters with:
 - The start character can be the underscore "_" or a capital or lower case letter
 - The letters following the start character can be anything which is permitted as a start character plus the digits
 - Identifiers are case-sensitive!
 - Python keywords are not allowed as identifier names!

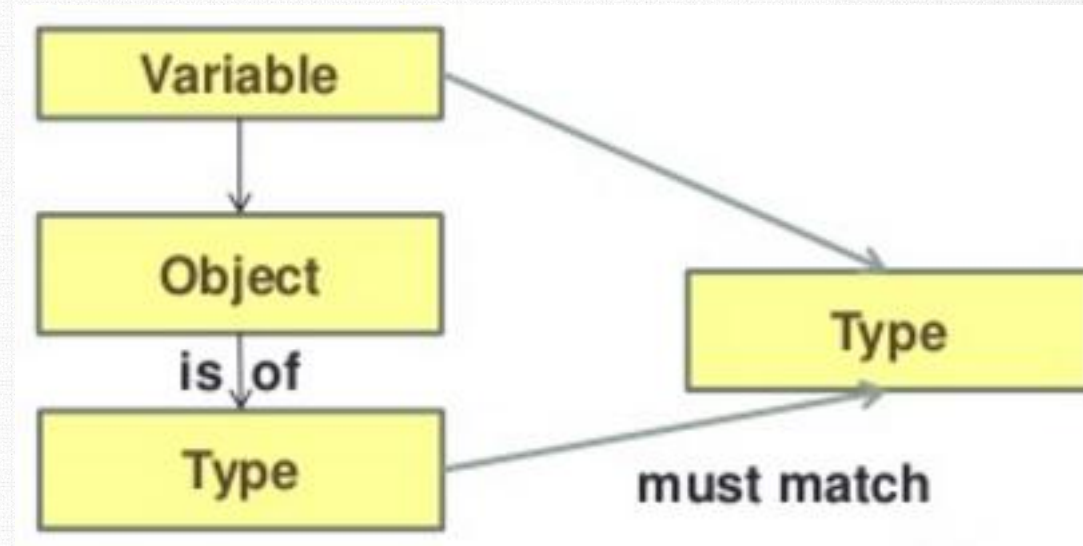
Python vs. other languages

Python



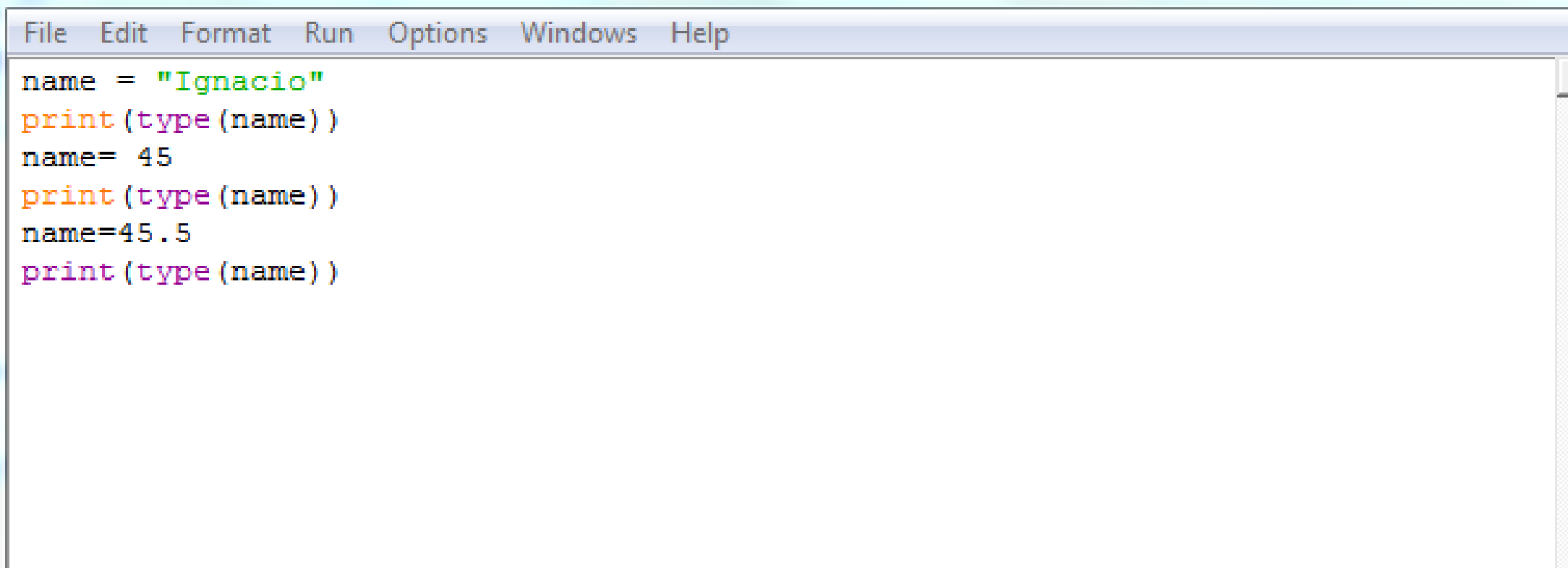
- Variable type determined at runtime
- Variable bound to one object and the object has only one type
- Variable can change type by changing the type of the object bound to the variable

Statically typed languages



- Bound to a type at compile time
- Bound to an object at runtime
- Need to declare the variable before using it

Python is a dynamically typed language

A screenshot of a Python IDE window. The menu bar at the top includes 'File', 'Edit', 'Format', 'Run', 'Options', 'Windows', and 'Help'. The code editor contains the following Python code:

```
name = "Ignacio"
print(type(name))
name= 45
print(type(name))
name=45.5
print(type(name))
```

The code demonstrates that the variable 'name' can hold different types of values (string, integer, and float) without any explicit type declaration or change, which is characteristic of a dynamically typed language.

Python is a strongly typed language

```
>>> var = 10
>>> var2 = '10'
>>> var3 = var + var2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
>>> _
```

With every great power...

- Guidelines

- Use descriptive names (x vs. sales_amount)
- Be consistent (user_name or userName?)
- Follow the traditions of the language
 - Usually in Python variable names start with a lowercase letter and avoid starting with an underscore
- Keep the length in check
 - no user_total_sales_month_report

Mathematical operators

Operator	Description	Example
+	Addition - Adds values on either side of the operator	$a + b$ will give 30
-	Subtraction - Subtracts right hand operand from left hand operand	$a - b$ will give -10
*	Multiplication - Multiplies values on either side of the operator	$a * b$ will give 200
/	Division - Divides left hand operand by right hand operand	b / a will give 2
%	Modulus - Divides left hand operand by right hand operand and returns remainder	$b \% a$ will give 0
**	Exponent - Performs exponential (power) calculation on operators	$a ** b$ will give 10 to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed.	$9 // 2$ is equal to 4 and $9.0 // 2.0$ is equal to 4.0

Mathematical operators

```
numbers.py - C:/Python34/numbers.py (3.5.1)
File Edit Format Run Options Window Help
a = 34
b = 8
c = 2

c = a + b
print ("Line 1 - Value of c is ", c)

c = a - b
print ("Line 2 - Value of c is ", c)

c = a * b
print ("Line 3 - Value of c is ", c )

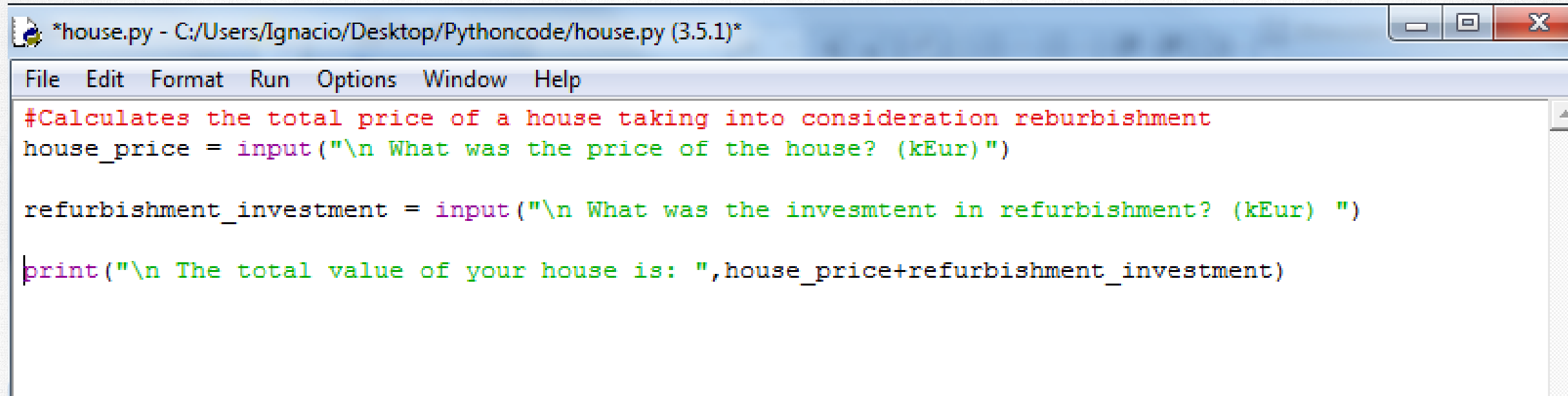
c = a / b
print ("Line 4 - Value of c is ", c )

c = a % b
print ("Line 5 - Value of c is ", c)

a = 2
b = 3
c = a**b
print ("Line 6 - Value of c is ", c)

a = 10
b = 5
c = a//b
print ("Line 7 - Value of c is ", c)
|
```

What is wrong with this program?



```
*house.py - C:/Users/Ignacio/Desktop/Pythoncode/house.py (3.5.1)*
File Edit Format Run Options Window Help
#Calculates the total price of a house taking into consideration reburbishment
house_price = input("\n What was the price of the house? (kEur)")

refurbishment_investment = input("\n What was the invesmtent in refurbishment? (kEur) ")

print("\n The total value of your house is: ",house_price+refurbishment_investment)
```

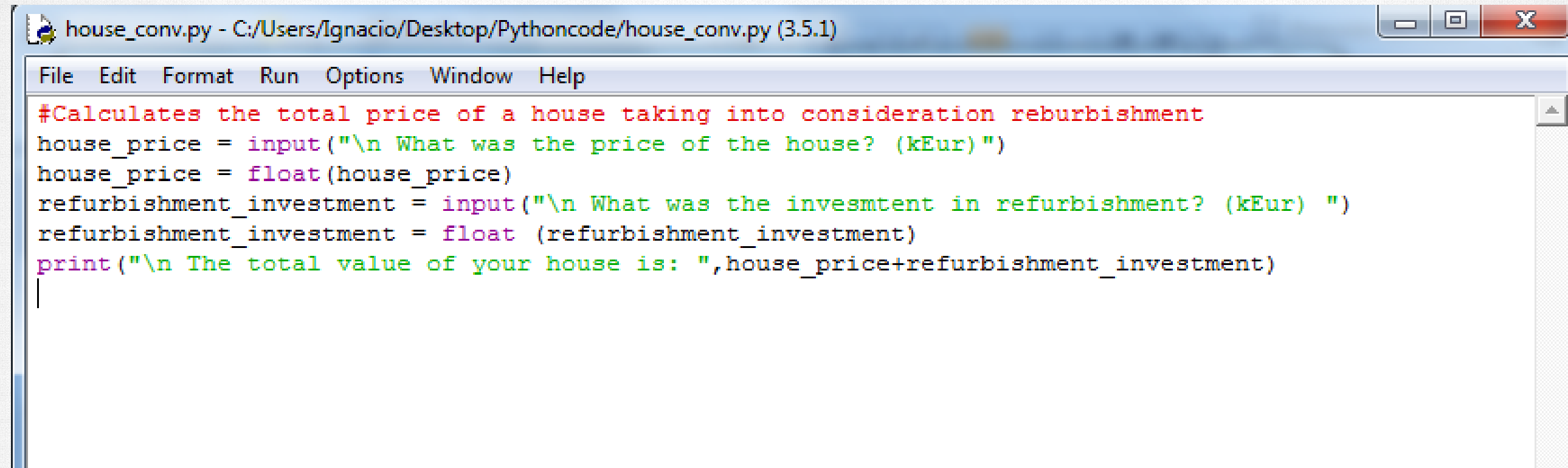
Converting values

- `float(x)` - returns a floating-point value by converting `x`
- `int(x)` - returns an integer value by converting `x`
- `str(x)` - returns a string value by converting `x`
- `bool(x)` – returns a boolean value

Data Type

Function	Description
<code>int(x [,base])</code>	Converts x to an integer. base specifies the base if x is a string.
<code>long(x [,base])</code>	Converts x to a long integer. base specifies the base if x is a string.
<code>float(x)</code>	Converts x to a floating-point number.
<code>complex(real [,imag])</code>	Creates a complex number.
<code>str(x)</code>	Converts object x to a string representation.
<code>repr(x)</code>	Converts object x to an expression string.
<code>eval(str)</code>	Evaluates a string and returns an object.
<code>tuple(s)</code>	Converts s to a tuple.
<code>list(s)</code>	Converts s to a list.
<code>set(s)</code>	Converts s to a set.
<code>dict(d)</code>	Creates a dictionary. d must be a sequence of (key,value) tuples.
<code>frozenset(s)</code>	Converts s to a frozen set.
<code>chr(x)</code>	Converts an integer to a character.
<code>unichr(x)</code>	Converts an integer to a Unicode character.
<code>ord(x)</code>	Converts a single character to its integer value.
<code>hex(x)</code>	Converts an integer to a hexadecimal string.
<code>oct(x)</code>	Converts an integer to an octal string.

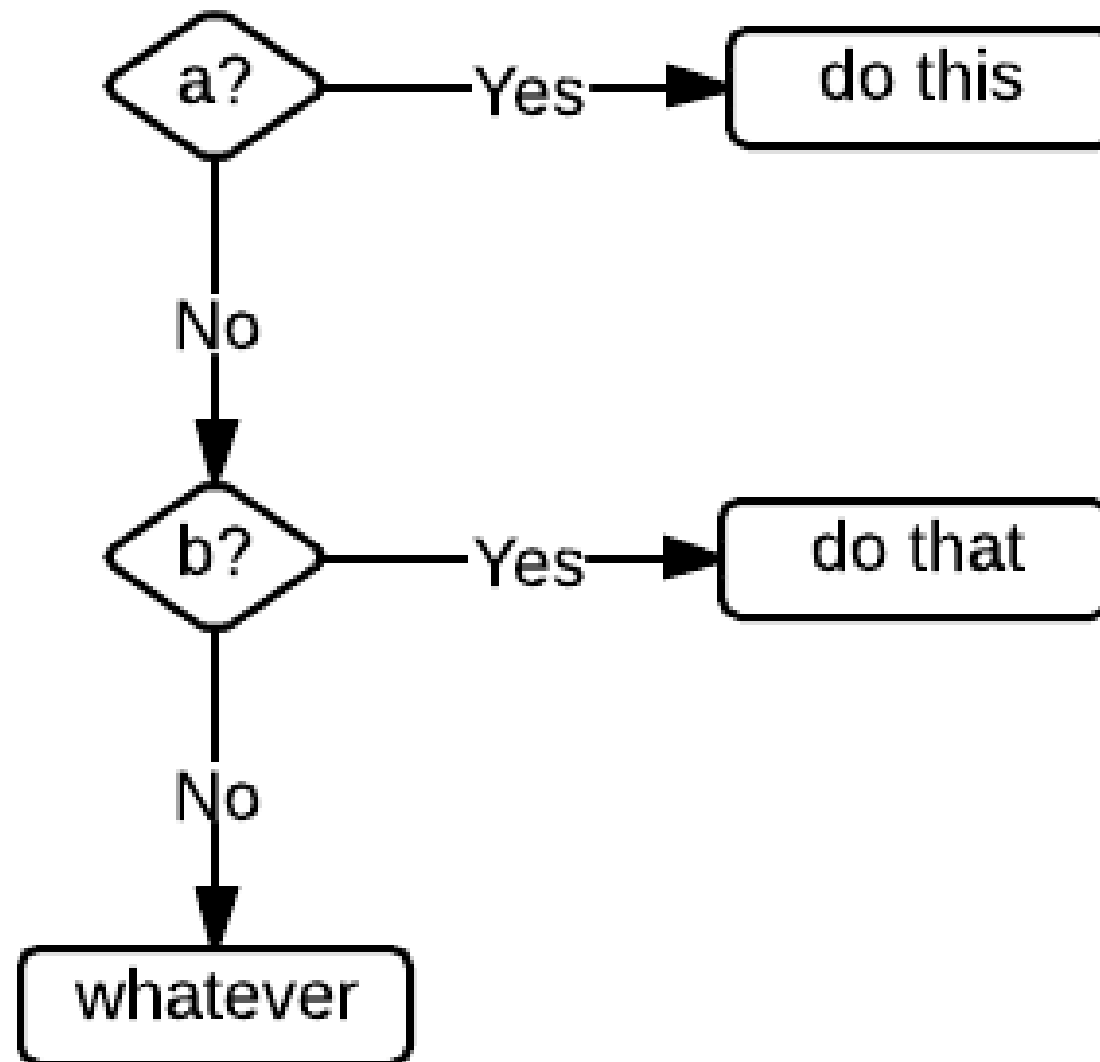
Converting values



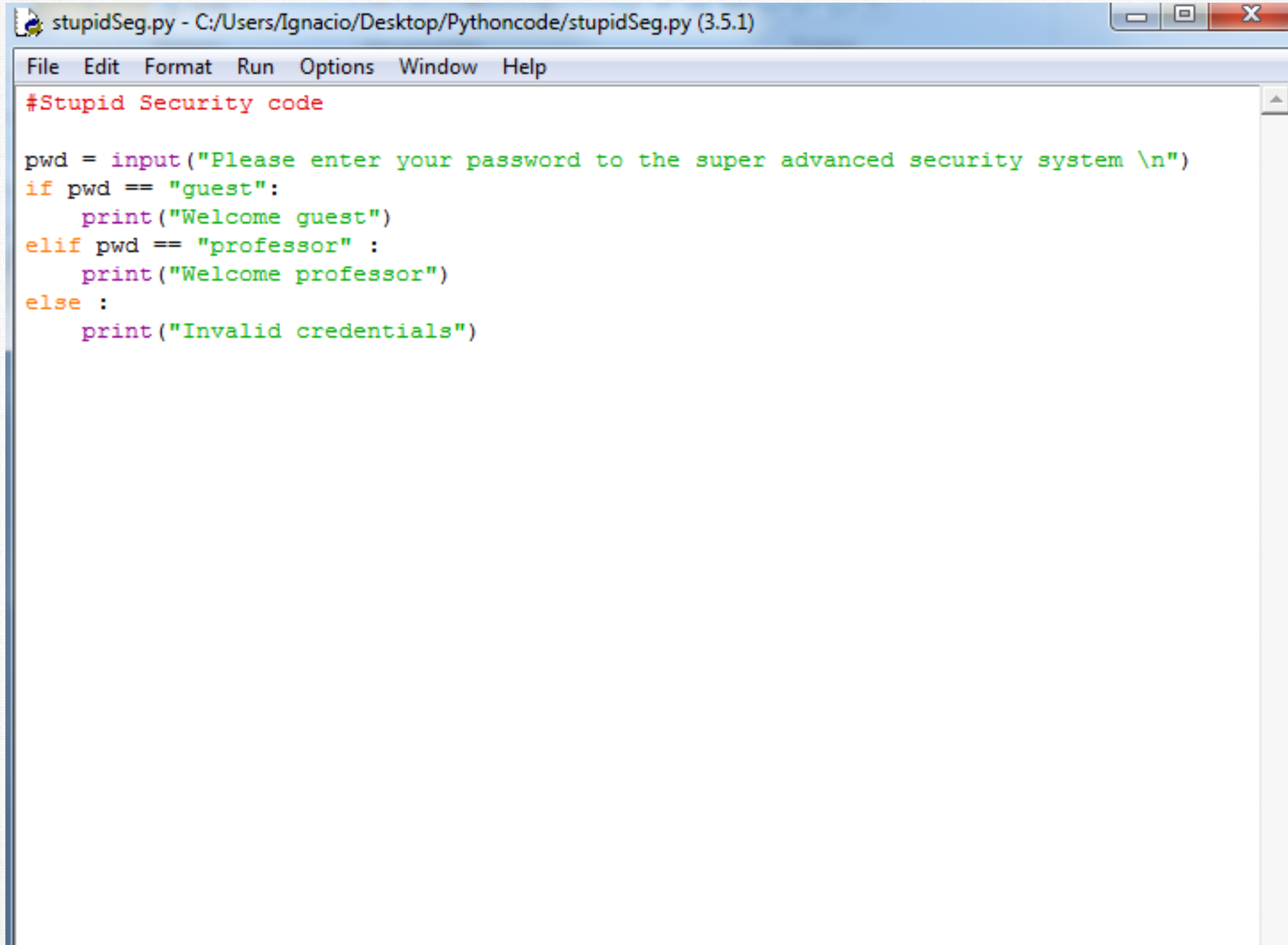
```
house_conv.py - C:/Users/Ignacio/Desktop/Pythoncode/house_conv.py (3.5.1)
File Edit Format Run Options Window Help
#Calculates the total price of a house taking into consideration reburbishment
house_price = input("\n What was the price of the house? (kEur)")
house_price = float(house_price)
refurbishment_investment = input("\n What was the invesmtent in refurbishment? (kEur) ")
refurbishment_investment = float (refurbishment_investment)
print("\n The total value of your house is: ",house_price+refurbishment_investment)
|
```

If – else - elif

```
if a:  
    do this  
elif b:  
    do that  
else:  
    whatever
```



If – else - elif

A screenshot of a Python IDE window titled "stupidSeg.py - C:/Users/Ignacio/Desktop/Pythoncode/stupidSeg.py (3.5.1)". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Window", and "Help". The code editor contains the following Python code:

```
#Stupid Security code

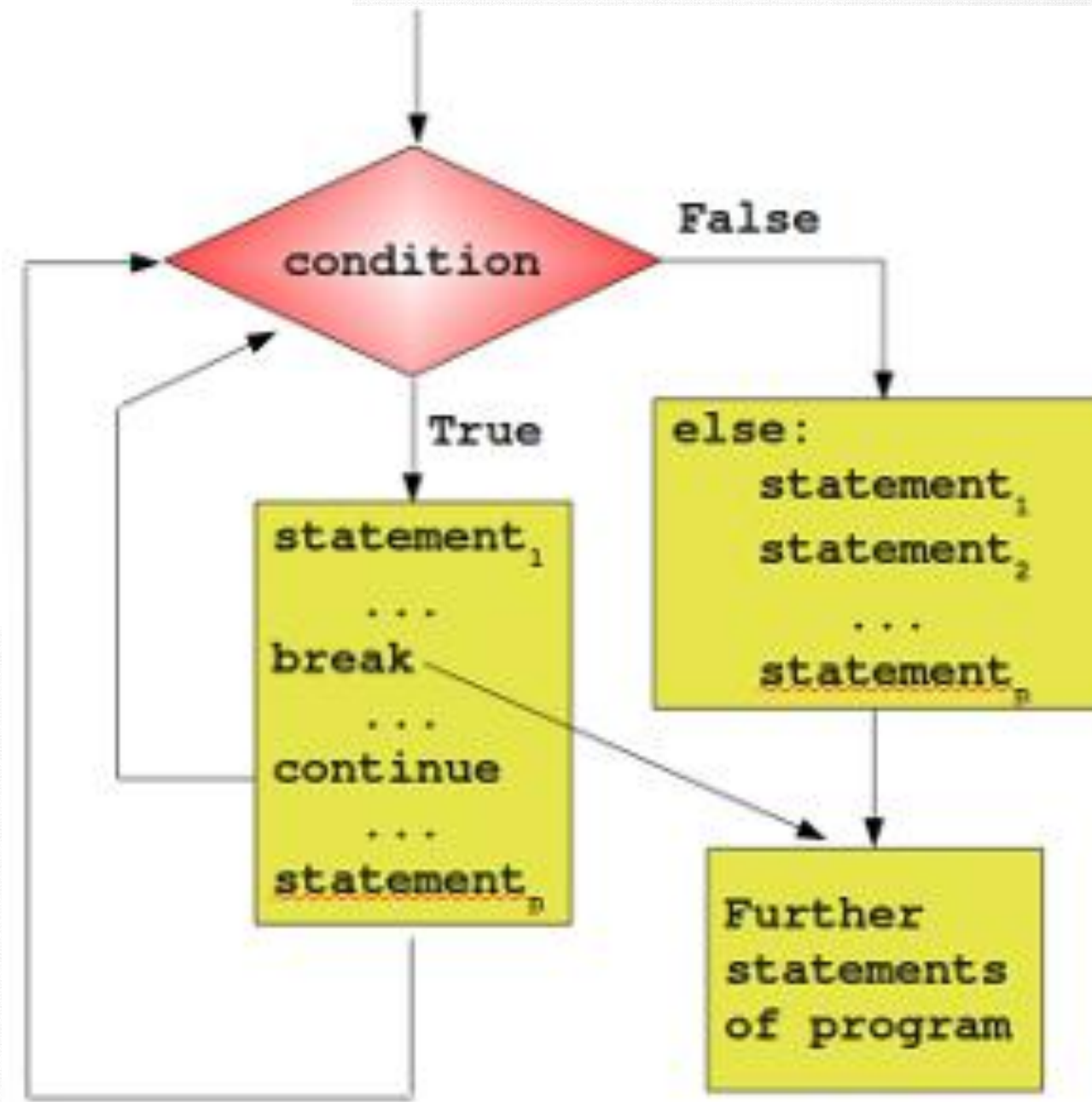
pwd = input("Please enter your password to the super advanced security system \n")
if pwd == "guest":
    print("Welcome guest")
elif pwd == "professor" :
    print("Welcome professor")
else :
    print("Invalid credentials")
```

Logical operators

Operator	Description	Operator	Description
a > b	Greater than	a and b	Logical AND If both operands are True then it returns True
a < b	Less than		
a >= b	Greater than or equal to	a or b	Logical OR If one of the operands is True then it returns True
a <= b	Less than or equal to		
a == b	Check if value of two operands is equal	not	Logical NOT
a != b	Check if value of two operands is not equal		

While Loop

```
flag = 0
while flag < 10 :
    print(flag)
    flag +=1
```



While and if

```
#Demonstrates logical operators and compound conditions

print("\t Super advanced security System")
print("\t\t Members Only! \n")

security = 0

username=""
#input in a while loop to force a non empty string input
while not username:
    username = input("Please input your username: ")

password = ""
#input in a while loop to force a non empty string input
while not password:
    password = input("Please input your password: ")

if username == "Juanjo" and password == "Casado" :
    print("hello boss!!")
    security = 5

elif username == "Bill" and password == "Gates" :
    print("show me the money!!!")
    security = 3

elif username == "guest" or password == "guest" :
    print("I don't know you")
    security =1

else:
    print("Get out of here")
    security =0
```

Misusing expressions as defaults for function arguments

```
>>> def foo(bar=[]):          # bar is optional and defaults to [] if not specified
...     bar.append("baz")      # but this line could be problematic, as we'll see...
...     return bar
```

```
>>> foo()
["baz"]
>>> foo()
["baz", "baz"]
>>> foo()
["baz", "baz", "baz"]
```

```
def foo(bar=None):
    if bar is None:           # or if not bar:
        bar = []
    bar.append("baz")
    return bar
```

Part 2: Troubleshooting/Debugging

Misunderstanding Python scope rules

```
token = 'global'

def access_local():
    token = 'local'
    if 'token' in locals() and 'token' in globals(): print("Yes, token is in both local
and global scope.")
    print("But value of token used is = (" + token + ")\n")

def access_global():
    if 'token' in globals(): print("Yes, token is in global scope.")
    print("Value of token used is = (" + token + ")\n")

def access_enclosed():
    test = 1
    for test in range(5):
        token = 'enclosed'
        pass
    if 'token' in globals(): print("Though, token is in global scope.")
    print("But value of token used is = (" + token + ")\n")

def id(token):
    return 1

access_local()
access_enclosed()
access_global()
print("%s = %d\n" % ("token length", id(token)))
print(token)
```

Yes, token is in both local and global scope.
But value of token used is = (local)

Though, token is in global scope.
But value of token used is = (enclosed)

Yes, token is in global scope.
Value of token used is = (global)

Not Using Exceptions At All

```
import os
# Testing the obvious, while overlooking the EAFP principle.
if os.path.exists("debug.log"):
    os.remove("debug.log")
```

What if “debug.log” exist, but there comes some error while removing it?

```
# - SOLUTION
import os
try:
    os.remove("debug.log")
#Raised when file isn't available.
except (Exception, OSError) as e:
    print (str(e))
```

Part 2: Troubleshooting/Debugging

Modifying a list while iterating over it

```
odd = lambda x : bool(x % 2)
numbers = [n for n in range(10)]
for i in range(len(numbers)):
    if odd(numbers[i]):
        del numbers[i] # BAD: Deleting item from a list while iterating over it
```

```
>>> odd = lambda x : bool(x % 2)
>>> numbers = [n for n in range(10)]
>>> numbers[:] = [n for n in numbers if not odd(n)] # ahh, the beauty of it all
>>> numbers
[0, 2, 4, 6, 8]
```

Part 2: Troubleshooting/Debugging

What if I don't know how to solve the problem:

- 1 - Try to interpret the error message/error logs.

```
import pandas_datareader as pdr  
  
ModuleNotFoundError: No module named 'pandas_datareader'
```

- 2- Search the web with the error message

```
pip install pandas_datareader
```

- 3 - Use an Idle (Spyder or Pycharm)
Open the variable explorer to check if the values of the variables make sense.

Part 2: Troubleshooting/Debugging

4 - Use debugging tools: <https://www.youtube.com/watch?v=Jb2HHOahvcE>

5 - Try to get to the line which the code is breaking by placing prints using “clever” numbering...

```
In [89]: def func1(a):
...:     print ("1.0")
...:     a = a + 1
...:     print ("1.1")
...:     return a
...:
...:
...:
...: def func2(a):
...:     print ("2.0")
...:     a = a + "a"
...:     print ("2.1")
...:     return a
...:
...:
...:
...: print(func1(1))
...: print(func2(1))
```

```
1.0
1.1
2
2.0
```

Part 2: Troubleshooting/Debugging

6 – If the error is inside the data, we can locate the data point with error by using binary search:

<http://www.geeksforgeeks.org/binary-search/>

```
import numpy as np
my_list_of_numbers = np.random.normal(size=1000000)
print(len(my_list_of_numbers))
print (sum(my_list_of_numbers))
my_list_of_numbers[544852] = None #INTRODUCING AN ERROR, CAN YOU FIND IT QUICK?
print(sum(my_list_of_numbers))

# Iterative Binary Search Function - It returns the position of the problem if it can find it, else returns -1
def binarySearchofError(myfunction, arr=None, l=0, r=None):
    try:
        value_wrong = myfunction(arr[l:r])
    except:
        value_wrong = ''
    k_depth = 0
    if r==None:
        r = len(arr)
    mid = int(l + (r - l)/2);
    while l <= r:
        k_depth += 1
        if r - l <= 1:
            return int((r+l)/2)
        mid = int(l + (r - l)/2);
        try:
            if (np.isnan(value_wrong) and np.isnan(myfunction(arr[mid:r]))) or (myfunction(arr[mid:r]) == value_wrong):
                print (k_depth,"RIGHT:",mid,r)
                l = mid #IF MATCHES THE ERROR VALUE, IT MEANS THAT THE ERROR IS BETWEEN mid AND r, so l BECOMES mid.
            else:
                print (k_depth,"LEFT:",l,mid)
                r = mid #IF NO ERROR AND IT DOES NOT MATCHES THE ERROR VALUE, IT MEANS THAT THE ERROR IS BETWEEN l AND mid, SO r BECOMES mid.
        except:
            print (k_depth,"RIGHT:",mid,r)
            l = mid #IF ERROR, IT MEANS THAT THE ERROR IS NOT BETWEEN mid AND r, so l BECOMES mid.
    return -1 # IF WE REACH HERE, WE WERE NOT ABLE TO FIND THE ERROR.

binarySearchofError(np.sum, arr=my_list_of_numbers)
```

Part 2: Troubleshooting/Debugging

7 – Restart Python, Notebook, Spyder whatever you are using

8 – Restart the Operational System

9 – Check/Update/Downgrade Python Package

10 – Change your code to use a different Python Package, library or data type

11 – Test a different Operational System, if it work somewhere else (stay there or check versions - Check/Update/Downgrade Python Package)

12 – Start coding again, consider doing everything from scratch.

Part 2: Troubleshooting/Debugging

Some references:

<https://www.toptal.com/python/top-10-mistakes-that-python-programmers-make>

<http://www.techbeamers.com/python-programming-mistakes/>